UNIVERSITE CATHOLIQUE D'AFRIQUE DE L'OUEST

UNITE UNIVERSITAIRE AU TOGO

ECOLE SUPERIEURE D'INGENIEURS

CYCLE INGENIEUR

NIVEAU : MASTER 1

TP : GROUPE 6

BIG DATA, HADOOP ET APPLICATIONS PYTHON

Hive et Data Warehouse distribué

ETUDIANTS :

AMAH-TCHOUTCHOUI Ekué Donald
FIOKLOU Donia
MINGANE Gracia
MOUN-RE Maxime

ENSEIGNANT :

Dr. Sena Apeke

ANNEE SCOLAIRE : 2025 -2026

Table des matières

Contexte et objectifs 1
Objectifs pédagogiques 1
Prérequis 1
Installation de l'environnement Docker : Windows 10 1
Étape 1 : Installer WSL2 1
Étape 2 : Installer Docker Desktop 2
Étape 3 : Récupérer les images Hadoop/Hive 3
Étape 4 :Charger le fichier dans HDFS 6
Étape 5 : Créer la table Hive 6
4. TP détaillé en Python (PyHive) 13
Étape 1 : Installer les dépendances Python sur la machine hôte 13
Étape 2 : Se connecter à Hive depuis Python 14
Étape 3 : Requêtes d'analyse demandées 14
Script :HiveQL complet 15
Resultat du TP 24
Dépannage courant 31

Contexte et objectifs

Après avoir manipulé HDFS (Exposé 2), MapReduce (Exposé 3) et Hadoop Streaming (Exposé 4), cet exposé introduit Hive, une couche d'abstraction déclarative permettant d'interroger des données stockées dans HDFS avec un langage proche du SQL (HiveQL). L'objectif est de construire une couche analytique SQL, un Data Warehouse distribué, au-dessus des données de transactions commerciales multisites (huit villes du Togo) utilisées comme fil conducteur du projet, puis de valider les résultats par rapport à ceux de MapReduce.

Objectifs pédagogiques

- Comprendre le rôle de Hive et du Metastore dans l'écosystème Hadoop (Hive stocke le schéma, HDFS stocke les données)
- Créer une table externe Hive pointant vers les fichiers CSV de HDFS, puis une organisation partitionnée pertinente (année, mois) au format ORC
- Écrire des requêtes HiveQL d'agrégation : chiffre d'affaires mensuel et annuel, panier moyen par ville, Top 10 produits, ventes par catégorie et mode de paiement, impact des promotions, activité par jour de semaine
- Piloter Hive depuis Python (PyHive) pour automatiser, exporter en CSV et visualiser l'analyse
- Comparer les résultats Hive aux résultats MapReduce, expliquer les éventuelles divergences et comparer les temps d'exécution

Prérequis

- Windows 10 version 2004 ou supérieure (build 19041+), 64 bits
- Virtualisation activée dans le BIOS/UEFI (Intel VT-x / AMD-V)
- Au moins 8 Go de RAM (4 Go alloués au cluster Docker recommandés)
- Python 3.9+ installé sur la machine hôte
- Un compte administrateur local pour l'installation de WSL2 et Docker Desktop

Installation de l'environnement Docker : Windows 10

Étape 1 : Installer WSL2

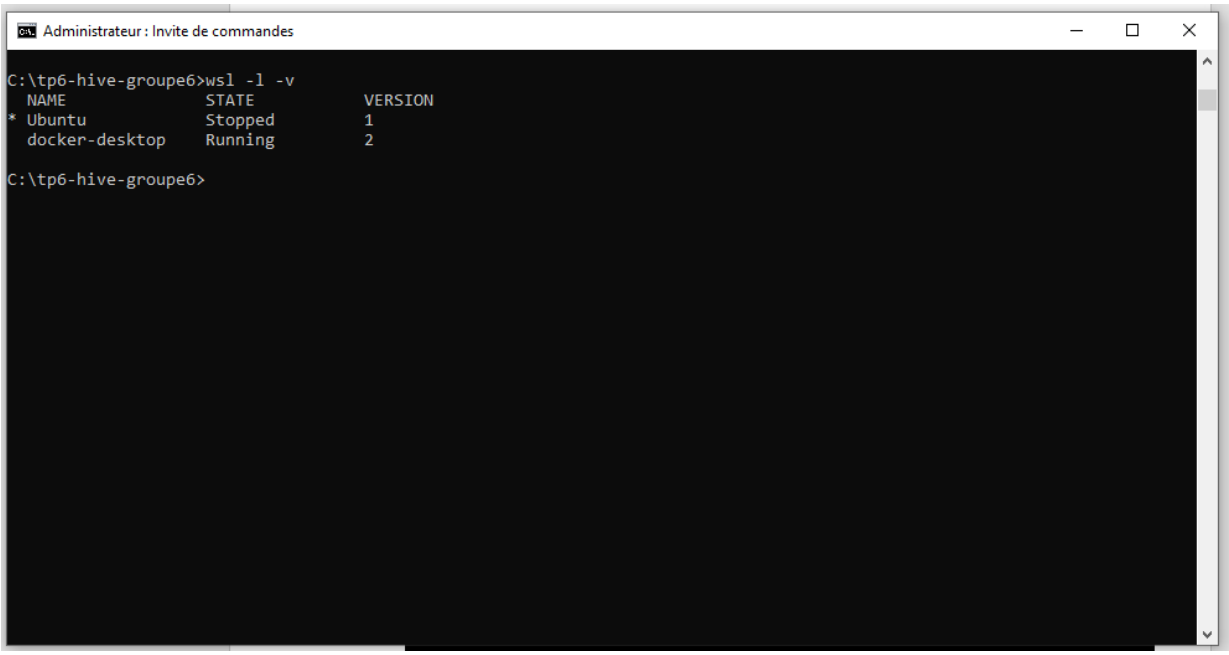
Ouvrir PowerShell en administrateur puis exécuter :

```
wsl --install

wsl --set-default-version 2
```

Redémarrer l'ordinateur si demandé. Vérifier la version installée :

```
wsl -l -v
```

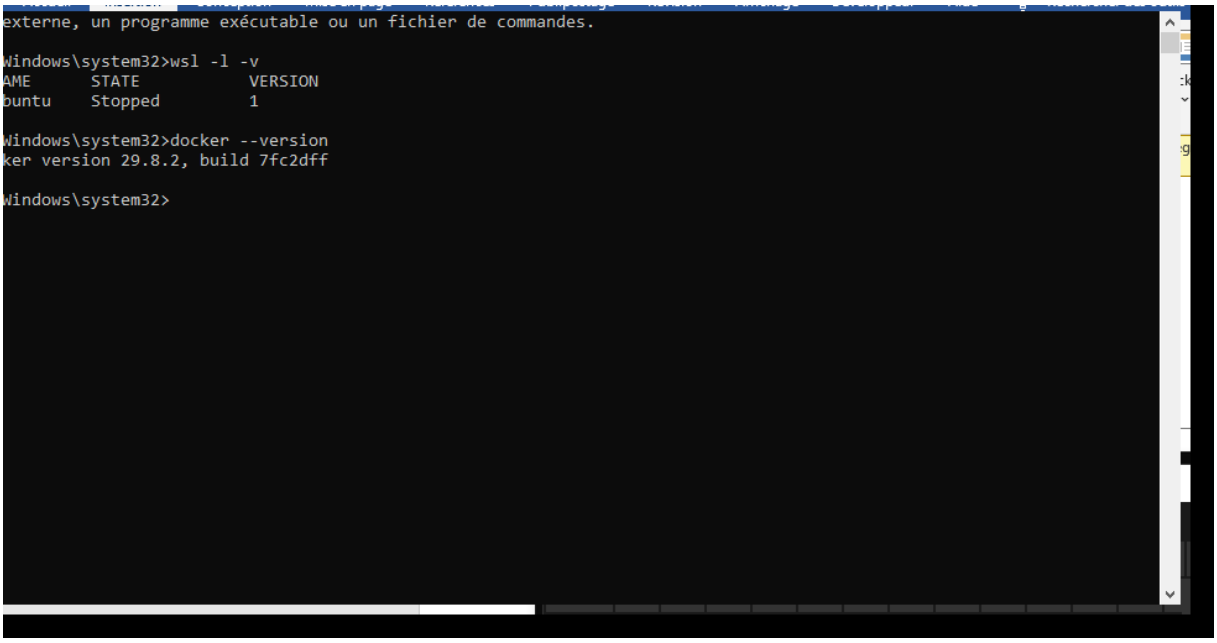


Étape 2 : Installer Docker Desktop

- Télécharger Docker Desktop pour Windows depuis docker.com/products/docker-desktop
- Lancer l'installeur et cocher « Use WSL 2 instead of Hyper-V »
- Redémarrer la machine, puis lancer Docker Desktop
- Dans Docker Desktop → Settings → Resources, allouer au moins 4 Go de RAM et 2 CPU

Vérifier l'installation dans PowerShell :

```
docker --version
docker compose version
```



<u>Étape 3</u> : Récupérer les images Hadoop/Hive

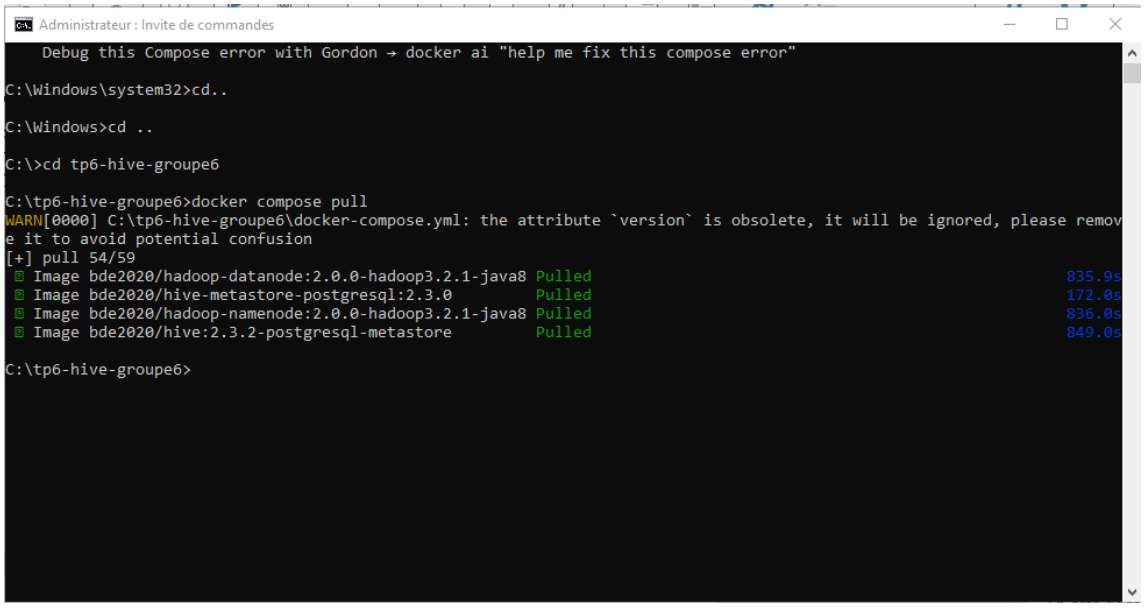
Créer un dossier de travail, par exemple C:\tp6-hive, et y placer les trois fichiers fournis:

Docker-compose.yml	Orchestration des conteneurs (namenode, datanode, hive server, metastore)
Hadoop-hive.env	Variables de configuration communes
Analyse_transactions.hql et tp6_hive_python.py	Scripts du TP
Transactions_small.csv Transactions_medium.csv	Jeu de test (10 000 lignes) Jeu de développement et de validation (500 000 lignes)

Dans PowerShell, se placer dans ce dossier puis télécharger les images et démarrer le

```
cd C:\tp6-hive
docker compose pull
docker compose up -d
```

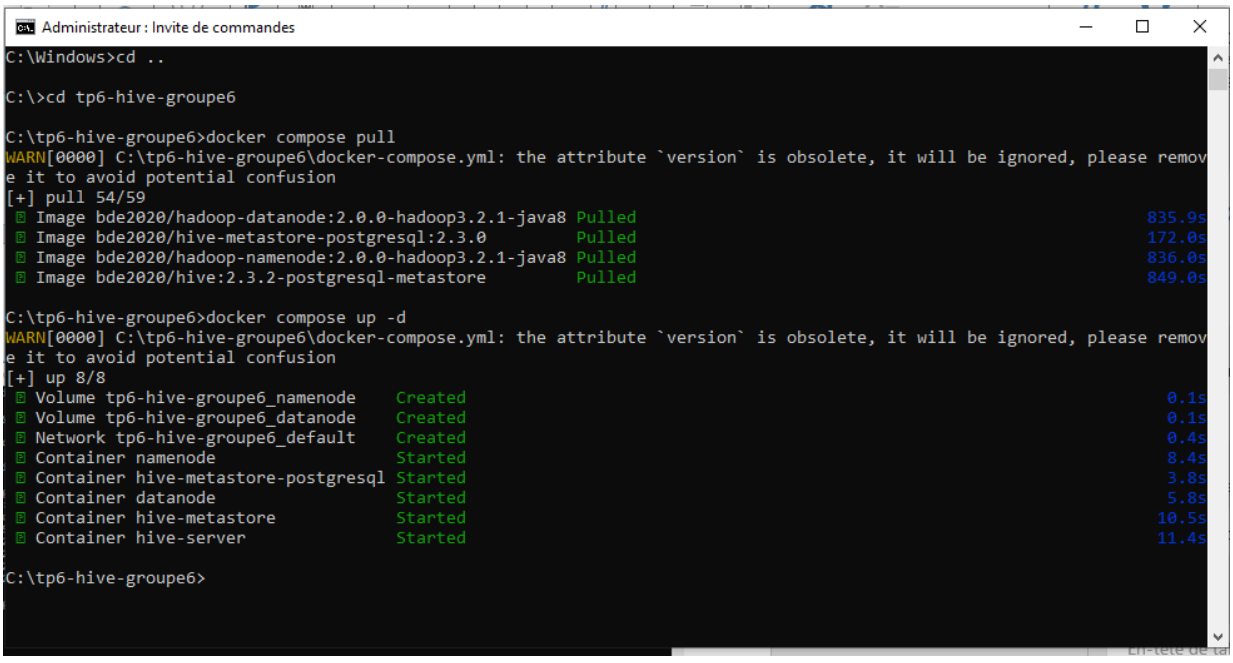
cluster :

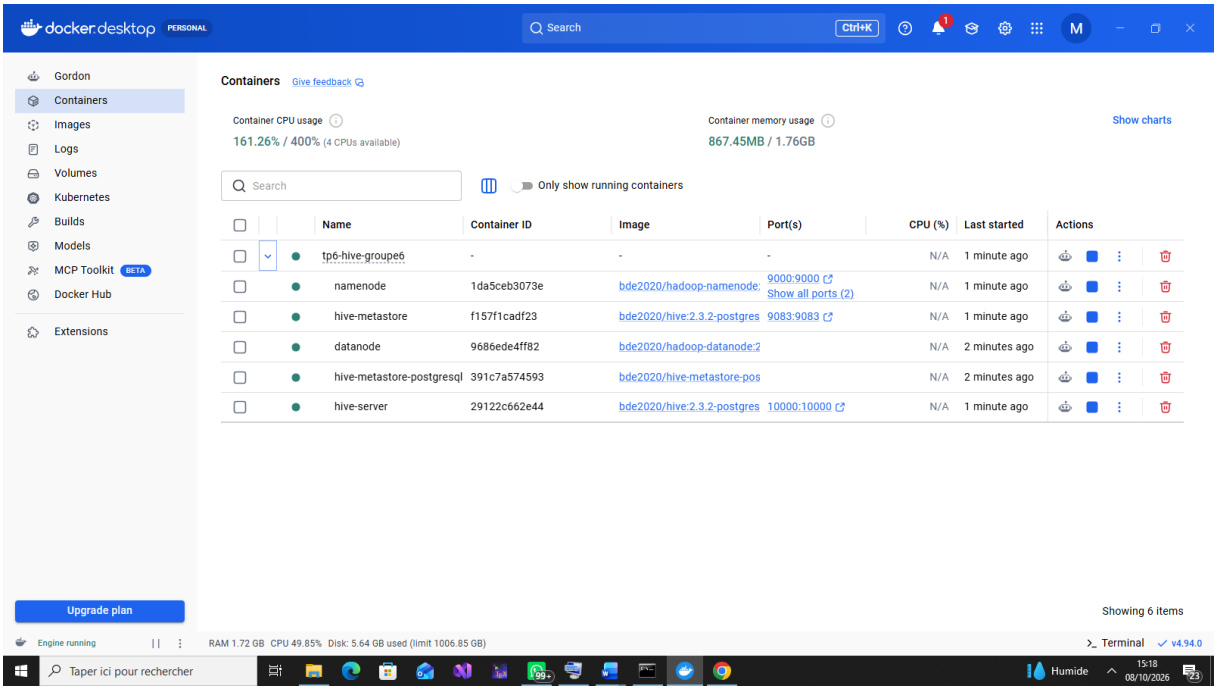


Le premier téléchargement peut prendre plusieurs minutes (plusieurs images de 1 Go). Vérifier que tous les conteneurs sont bien démarrés :

```
docker ps
```

On doit voir 5 conteneurs actifs : namenode, datanode, hive-server, hive-metastore et hive-metastore-postgresql. L'interface web du NameNode est accessible sur http://localhost:9870





hadoop-namenode (Port 9870 / 9000)	C'est le cerveau de Hadoop. C'est lui qui sait où sont les fichiers.
hadoop-datanode (Port 9864)	C'est le disque dur. C'est là que les données sont vraiment stockées.
hive-metastore-postgresql (Port 5432)	C'est la petite base de données qui retient la structure de tes tables Hive.
hive-metastore (Port 9083)	C'est le catalogue. Il fait le lien entre Hive et la base PostgreSQL.
hive-server (Port 10000)	C'est le plus important ; C'est le serveur Hive où on va lancer les requêtes SQL.

Étape 4 :Charger le fichier dans HDFS

```
docker cp transactions_small.csv namenode:/transactions_small.csv

docker cp transactions_medium.csv namenode:/transactions_medium.csv

docker exec -it namenode hdfs dfs -mkdir -p /datalake/raw/transactions/small

docker exec -it namenode hdfs dfs -mkdir -p /datalake/raw/transactions/medium

docker exec -it namenode hdfs dfs -mkdir -p /datalake/results/hive

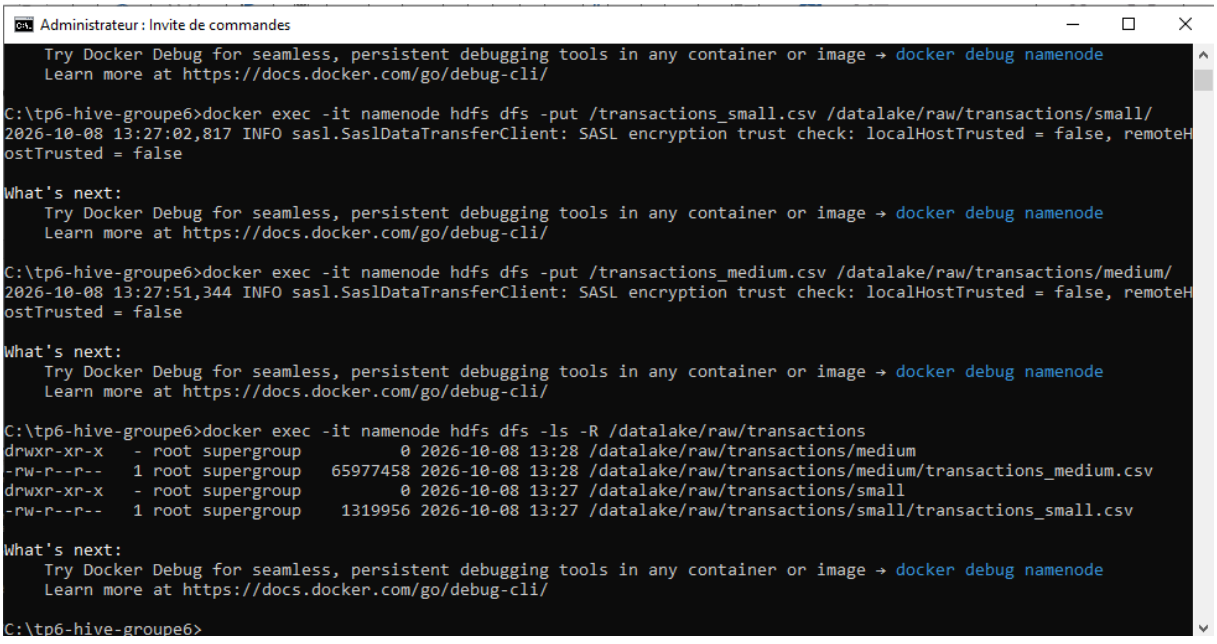
docker exec -it namenode hdfs dfs -put /transactions_small.csv
/datalake/raw/transactions/small/

docker exec -it namenode hdfs dfs -put /transactions_medium.csv
/datalake/raw/transactions/medium/

docker exec -it namenode hdfs dfs -ls -R /datalake/raw/transactions
```

Un répertoire par fichier : une table externe Hive lit tous les fichiers du répertoire indiqué par LOCATION. Mettre small et medium dans le même répertoire ferait compter 510 000 lignes au lieu de 500 000.

Si le Data Lake existe déjà, vérifier ses chemins avec la dernière commande avant de charger quoi que ce soit : toute création ou correction doit être documentée dans le README.md (règle de transmission).

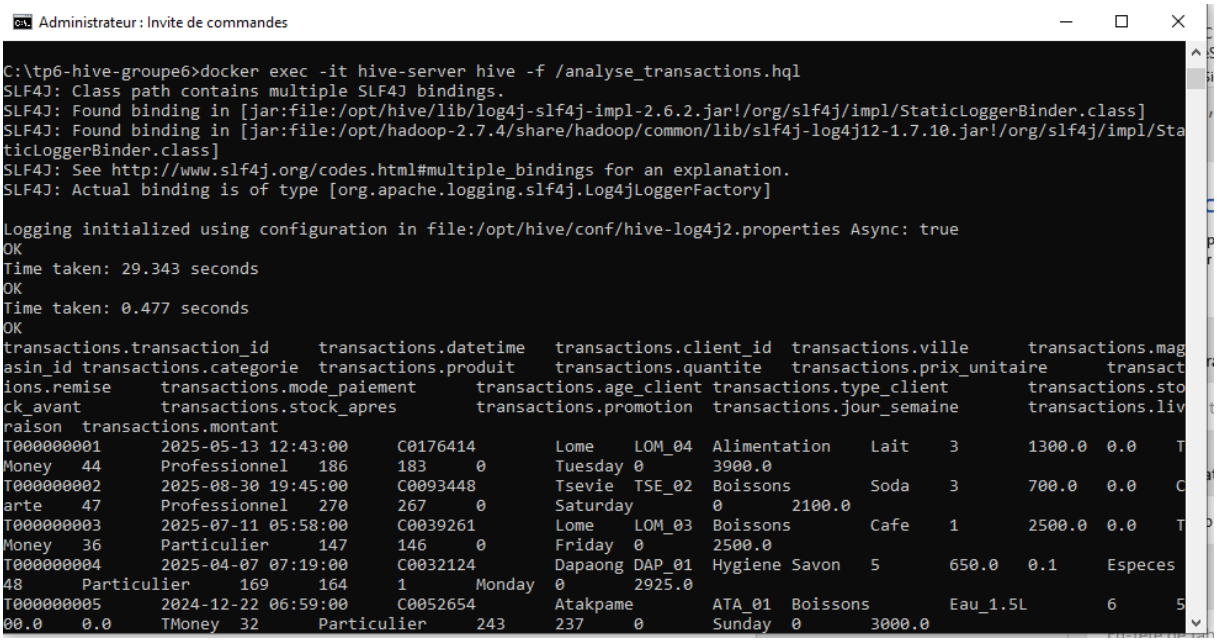


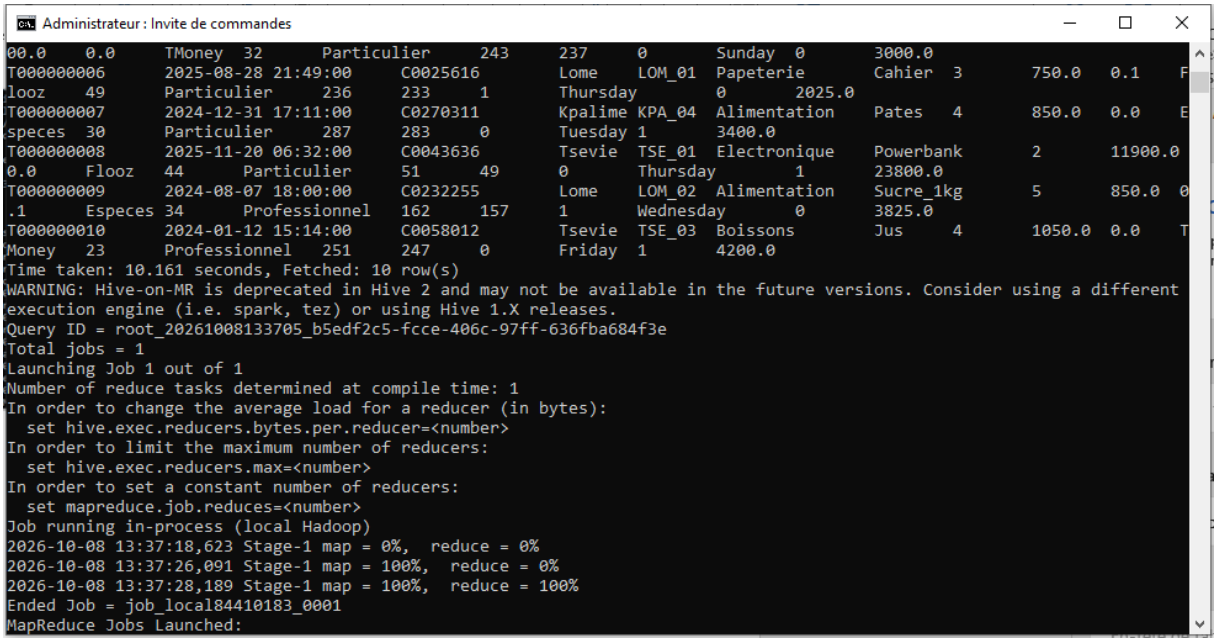
Étape 5 : Créer la table Hive

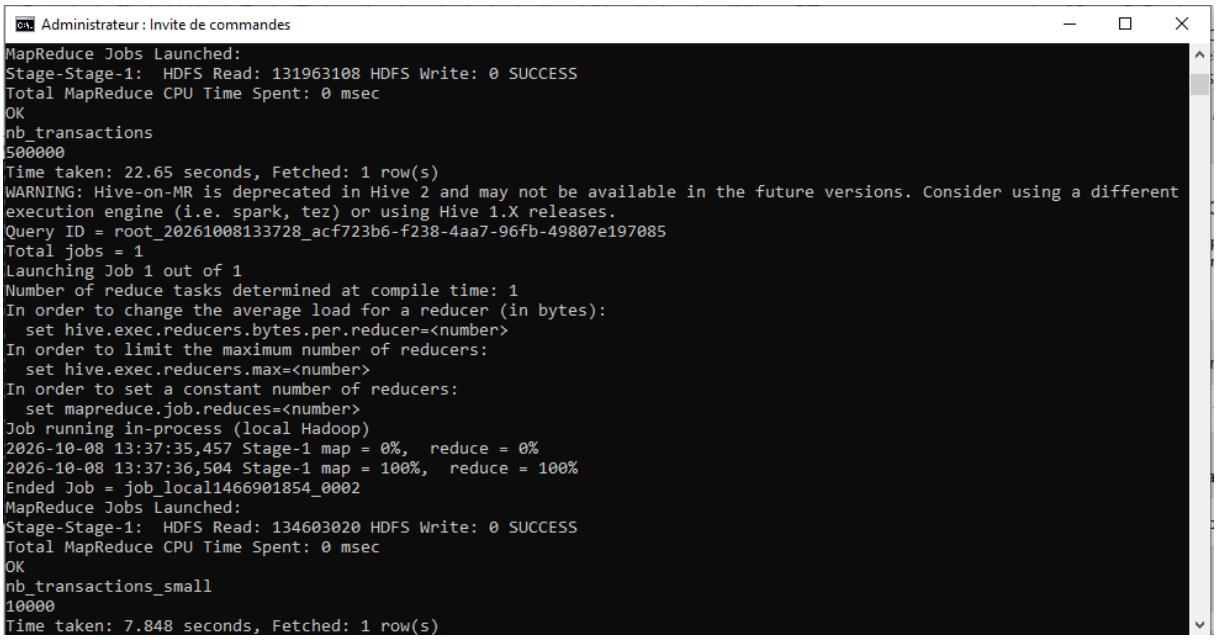
Copier le script analyse_transactions.hql dans le conteneur hive-server puis l'exécuter (ce script crée les tables externes, la table partitionnée ORC, puis lance les requêtes d'analyse ; compter plusieurs minutes pour le chargement des 24 partitions) :

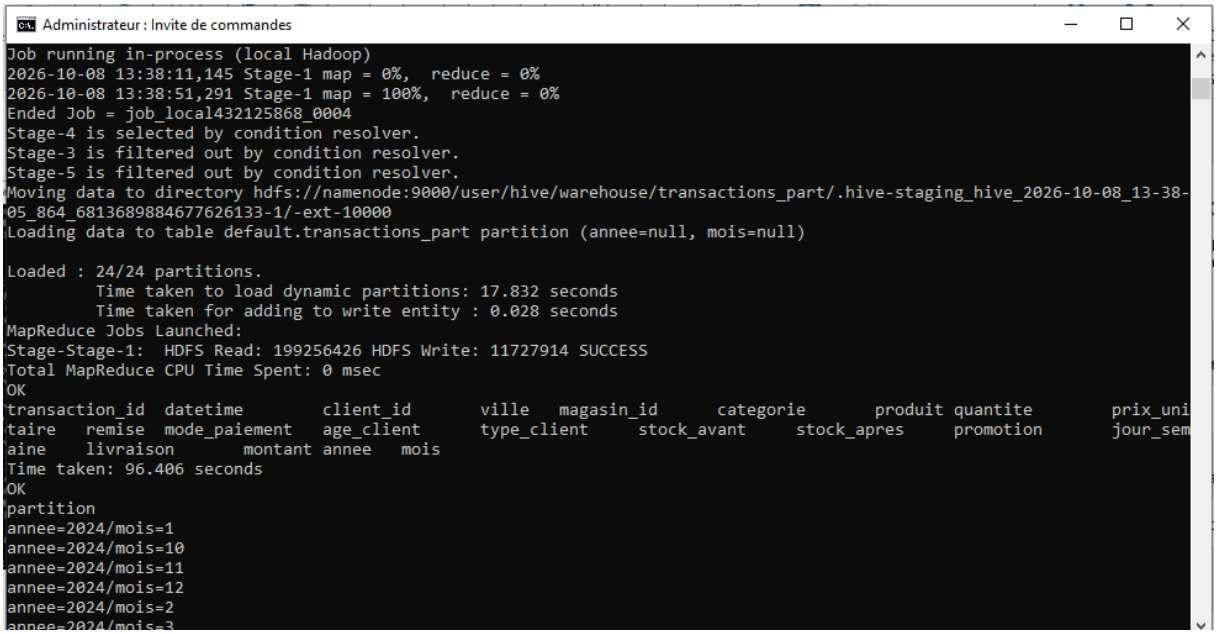
```
docker cp analyse_transactions.hql hive-server:/analyse_transactions.hql

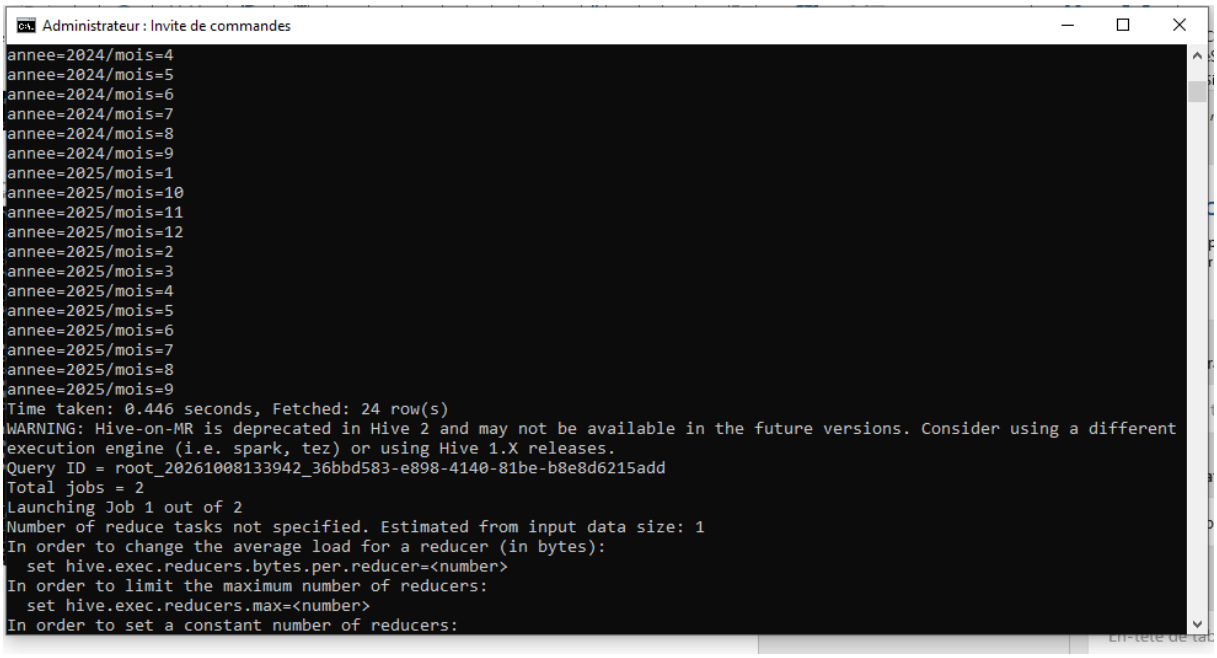
docker exec -it hive-server hive -f /analyse_transactions.hql
```

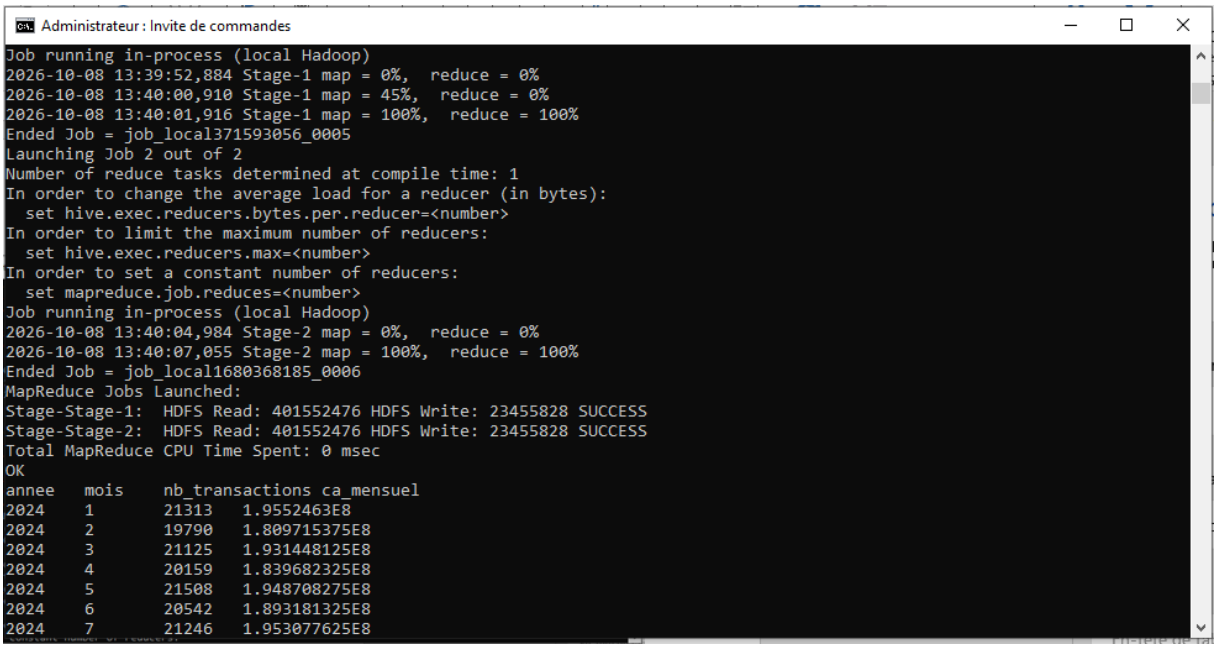


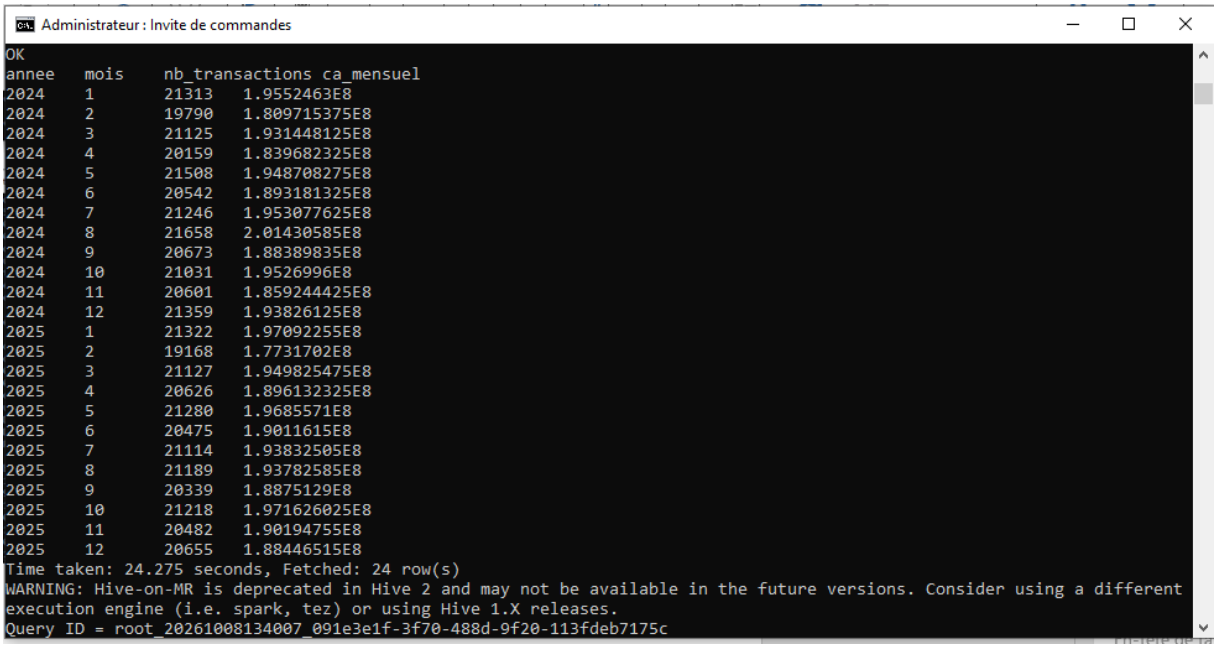


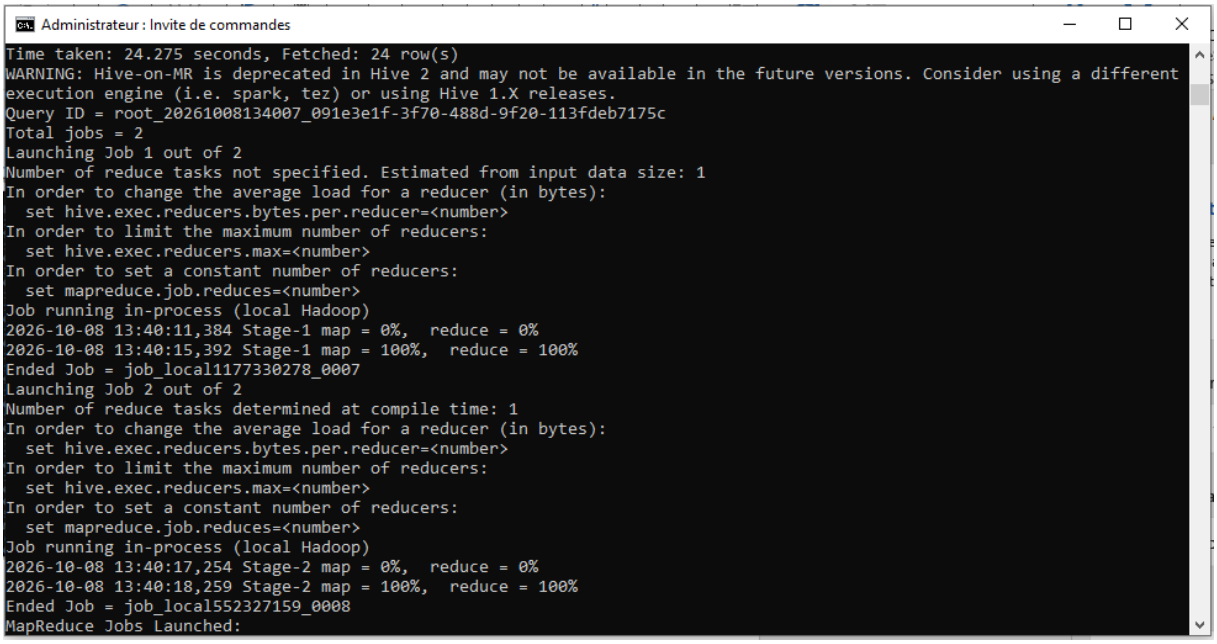

Administrateur : Invite de commandes
MapReduce Jobs Launched:
Stage-Stage-1: HDFS Read: 131963108 HDFS Write: 0 SUCCESS
Total MapReduce CPU Time Spent: 0 msec
OK
nb_transactions
500000
Time taken: 22.65 seconds, Fetched: 1 row(s)
WARNING: Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider using a different execution engine (i.e. spark, tez) or using Hive 1.X releases.
Query ID = root_20261008133728_acf723b6-f238-4aa7-96fb-49807e197085
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Job running in-process (local Hadoop)
2026-10-08 13:37:35,457 Stage-1 map = 0%, reduce = 0%
2026-10-08 13:37:36,504 Stage-1 map = 100%, reduce = 100%
Ended Job = job_local1466901854_0002
MapReduce Jobs Launched:
Stage-Stage-1: HDFS Read: 134603020 HDFS Write: 0 SUCCESS
Total MapReduce CPU Time Spent: 0 msec
OK
nb_transactions_small
10000
Time taken: 7.848 seconds, Fetched: 1 row(s)

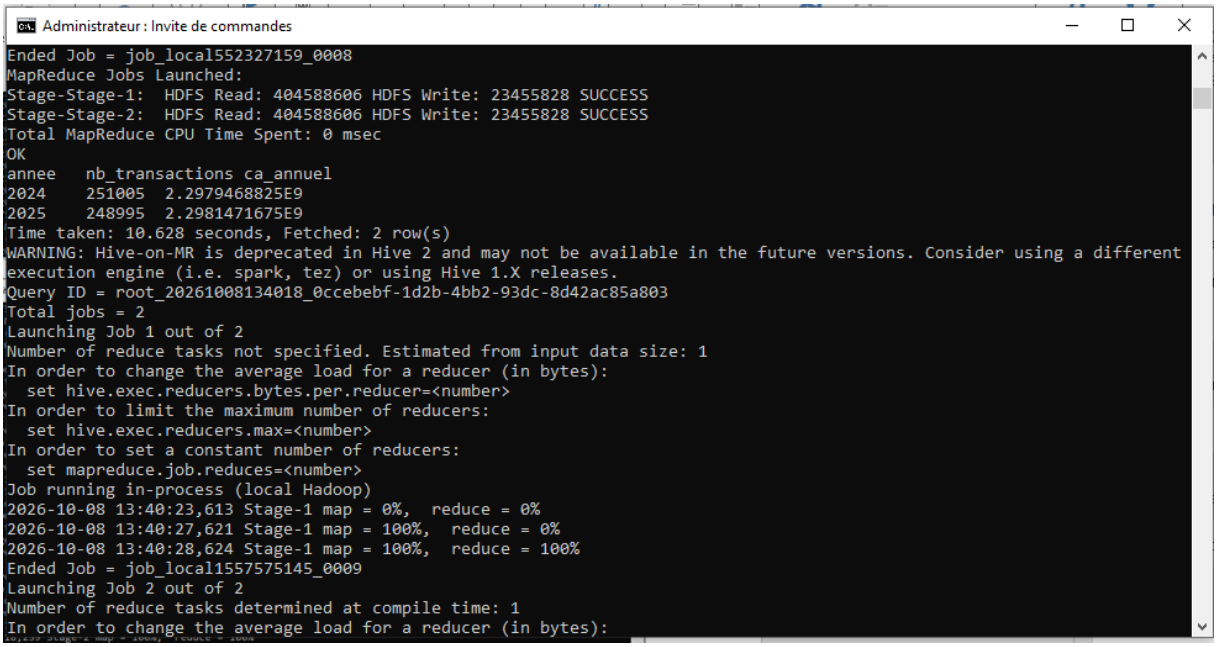

Administrateur : Invite de commandes
Job running in-process (local Hadoop)
2026-10-08 13:38:11,145 Stage-1 map = 0%, reduce = 0%
2026-10-08 13:38:51,291 Stage-1 map = 100%, reduce = 0%
Ended Job = job_local432125868_0004
Stage-4 is selected by condition resolver.
Stage-3 is filtered out by condition resolver.
Stage-5 is filtered out by condition resolver.
Moving data to directory hdfs://namenode:9000/user/hive/warehouse/transactions_part/.hive-staging_hive_2026-10-08_13-38-05_864_6813689884677626133-1/-ext-10000
Loading data to table default.transactions_part partition (annee=null, mois=null)
Loaded : 24/24 partitions.
Time taken to load dynamic partitions: 17.832 seconds
Time taken for adding to write entity : 0.028 seconds
MapReduce Jobs Launched:
Stage-Stage-1: HDFS Read: 199256426 HDFS Write: 11727914 SUCCESS
Total MapReduce CPU Time Spent: 0 msec
OK
transaction_id datetime client_id ville magasin_id categorie produit quantite prix_unitaire remise mode_paiement age_client type_client stock_avant stock_apres promotion jour_semaine livraison montant annee mois
Time taken: 96.406 seconds
OK
partition
annee=2024/mois=1
annee=2024/mois=10
annee=2024/mois=11
annee=2024/mois=12
annee=2024/mois=2
annee=2024/mois=3

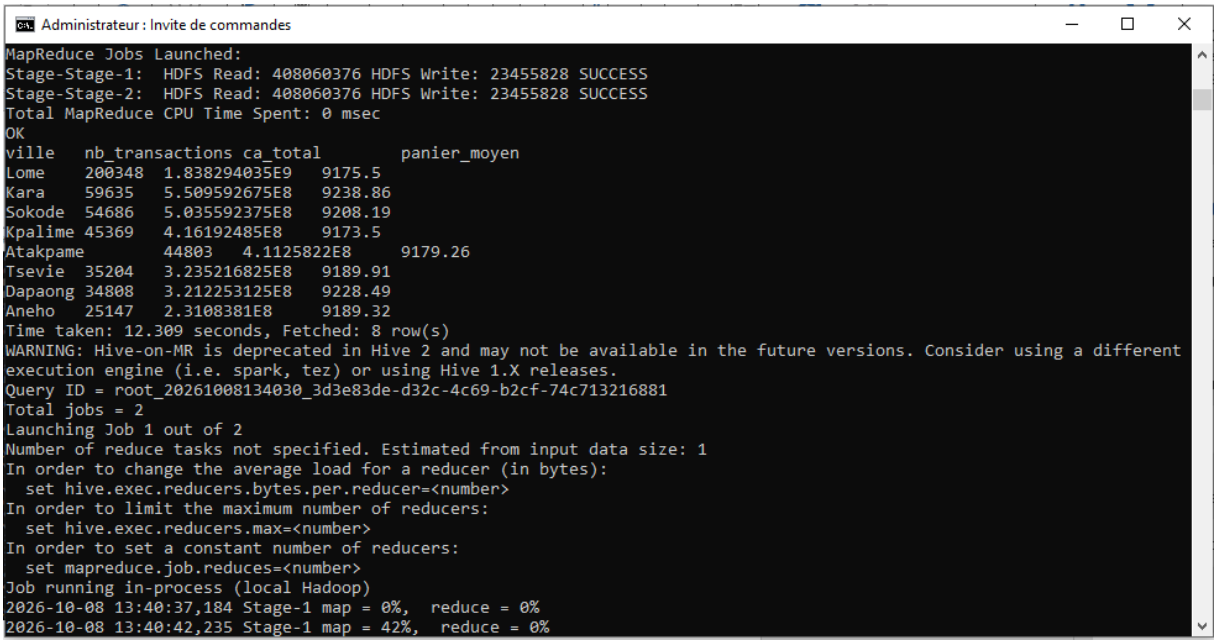

Administrateur : Invite de commandes
annee=2024/mois=4
annee=2024/mois=5
annee=2024/mois=6
annee=2024/mois=7
annee=2024/mois=8
annee=2024/mois=9
annee=2025/mois=1
annee=2025/mois=10
annee=2025/mois=11
annee=2025/mois=12
annee=2025/mois=2
annee=2025/mois=3
annee=2025/mois=4
annee=2025/mois=5
annee=2025/mois=6
annee=2025/mois=7
annee=2025/mois=8
annee=2025/mois=9
Time taken: 0.446 seconds, Fetched: 24 row(s)
WARNING: Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider using a different execution engine (i.e. spark, tez) or using Hive 1.X releases.
Query ID = root_20261008133942_36bbd583-e898-4140-81be-b8e8d6215add
Total jobs = 2
Launching Job 1 out of 2
Number of reduce tasks not specified. Estimated from input data size: 1
In order to change the average load for a reducer (in bytes):
set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:

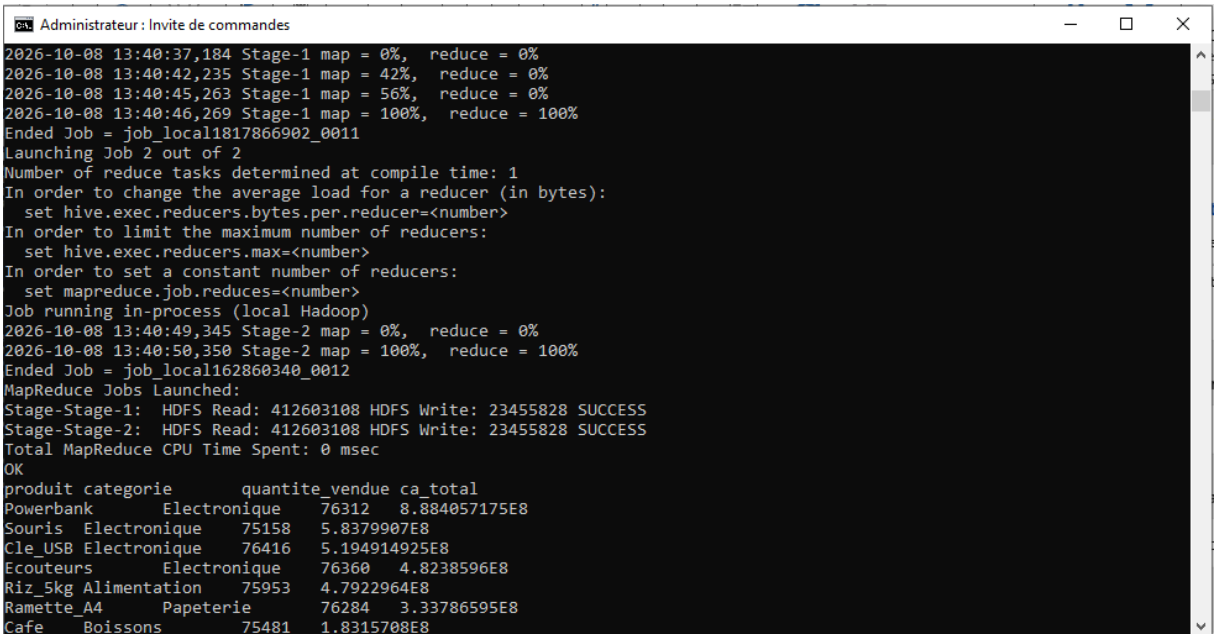

Administrateur : Invite de commandes
Job running in-process (local Hadoop)
2026-10-08 13:39:52,884 Stage-1 map = 0%, reduce = 0%
2026-10-08 13:40:00,910 Stage-1 map = 45%, reduce = 0%
2026-10-08 13:40:01,916 Stage-1 map = 100%, reduce = 100%
Ended Job = job_local371593056_0005
Launching Job 2 out of 2
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
set mapreduce.job.reduces=<number>
Job running in-process (local Hadoop)
2026-10-08 13:40:04,984 Stage-2 map = 0%, reduce = 0%
2026-10-08 13:40:07,055 Stage-2 map = 100%, reduce = 100%
Ended Job = job_local1680368185_0006
MapReduce Jobs Launched:
Stage-Stage-1: HDFS Read: 401552476 HDFS Write: 23455828 SUCCESS
Stage-Stage-2: HDFS Read: 401552476 HDFS Write: 23455828 SUCCESS
Total MapReduce CPU Time Spent: 0 msec
OK
annee mois nb_transactions ca_mensuel
2024 1 21313 1.9552463E8
2024 2 19790 1.809715375E8
2024 3 21125 1.931448125E8
2024 4 20159 1.839682325E8
2024 5 21508 1.948708275E8
2024 6 20542 1.893181325E8
2024 7 21246 1.953077625E8

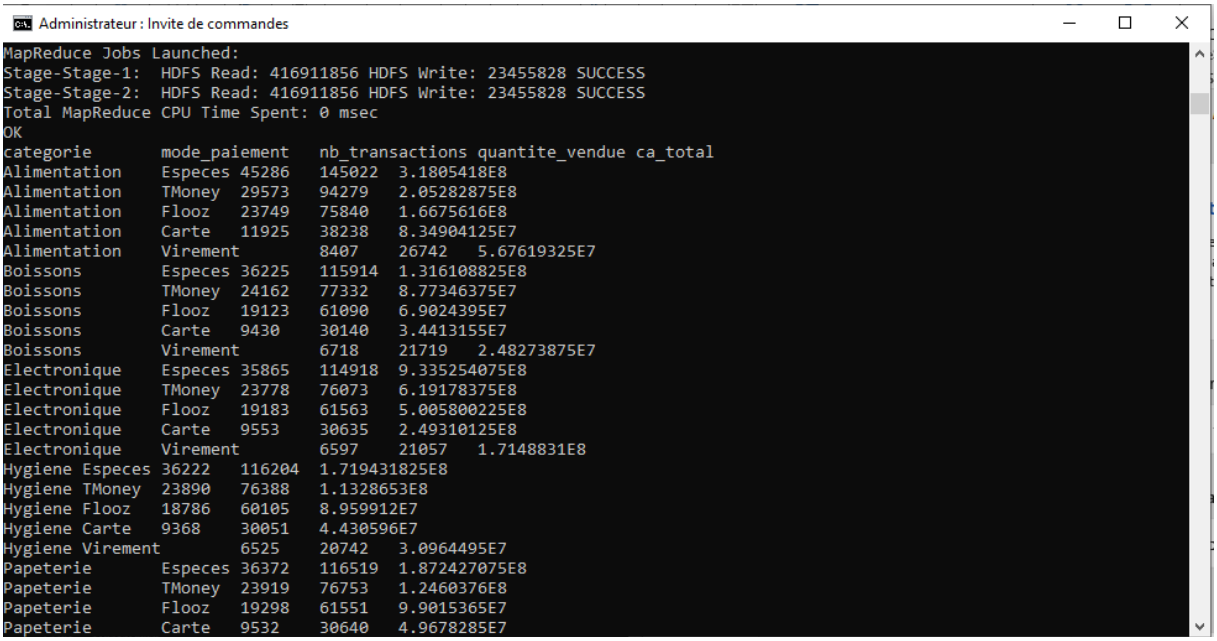

Administrateur : Invite de commandes
OK
annee mois nb_transactions ca_mensuel
2024 1 21313 1.9552463E8
2024 2 19790 1.80971537 5E8
2024 3 21125 1.931448125E8
2024 4 20159 1.839682325E8
2024 5 21508 1.948708275E8
2024 6 20542 1.893181325E8
2024 7 21246 1.953077625E8
2024 8 21658 2.01430585E8
2024 9 20673 1.88389835E8
2024 10 21031 1.9526996E8
2024 11 20601 1.859244425E8
2024 12 21359 1.93826125E8
2025 1 21322 1.97092255E8
2025 2 19168 1.7731702E8
2025 3 21127 1.949825475E8
2025 4 20626 1.896132325E8
2025 5 21280 1.9685571E8
2025 6 20475 1.9011615E8
2025 7 21114 1.93832505E8
2025 8 21189 1.93782585E8
2025 9 20339 1.8875129E8
2025 10 21218 1.971626025E8
2025 11 20482 1.90194755E8
2025 12 20655 1.88446515E8
Time taken: 24.275 seconds, Fetched: 24 row(s)
WARNING: Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider using a different
execution engine (i.e. spark, tez) or using Hive 1.X releases.
Query ID = root_20261008134007_091e3e1f-3f70-488d-9f20-113fdeb7175c

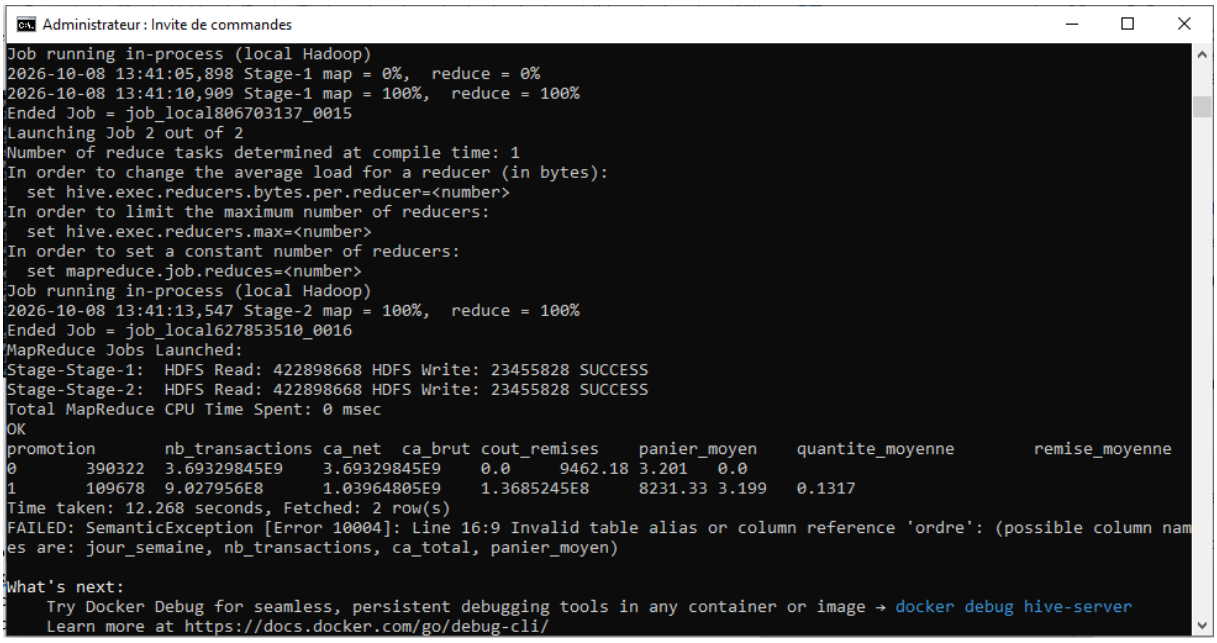

Administrateur : Invite de commandes
Time taken: 24.275 seconds, Fetched: 24 row(s)
WARNING: Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider using a different
execution engine (i.e. spark, tez) or using Hive 1.X releases.
Query ID = root_20261008134007_091e3e1f-3f70-488d-9f20-113fdeb7175c
Total jobs = 2
Launching Job 1 out of 2
Number of reduce tasks not specified. Estimated from input data size: 1
In order to change the average load for a reducer (in bytes):
set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
set mapreduce.job.reduces=<number>
Job running in-process (local Hadoop)
2026-10-08 13:40:11,384 Stage-1 map = 0%, reduce = 0%
2026-10-08 13:40:15,392 Stage-1 map = 100%, reduce = 100%
Ended Job = job_local1177330278_0007
Launching Job 2 out of 2
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
set mapreduce.job.reduces=<number>
Job running in-process (local Hadoop)
2026-10-08 13:40:17,254 Stage-2 map = 0%, reduce = 0%
2026-10-08 13:40:18,259 Stage-2 map = 100%, reduce = 100%
Ended Job = job_local552327159_0008
MapReduce Jobs Launched:


Administrateur : Invite de commandes
Ended Job = job_local552327159_0008
MapReduce Jobs Launched:
Stage-Stage-1: HDFS Read: 404588606 HDFS Write: 23455828 SUCCESS
Stage-Stage-2: HDFS Read: 404588606 HDFS Write: 23455828 SUCCESS
Total MapReduce CPU Time Spent: 0 msec
OK
annee	nb_transactions	ca_annuel
2024	251005	2.29794682SE9
2025	248995	2.298147167SE9
Time taken: 10.628 seconds, Fetched: 2 row(s)
WARNING: Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider using a different execution engine (i.e. spark, tez) or using Hive 1.X releases.
Query ID = root_20261008134018_0ccebebf-1d2b-4bb2-93dc-8d42ac85a803
Total jobs = 2
Launching Job 1 out of 2
Number of reduce tasks not specified. Estimated from input data size: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Job running in-process (local Hadoop)
2026-10-08 13:40:23,613 Stage-1 map = 0%, reduce = 0%
2026-10-08 13:40:27,621 Stage-1 map = 100%, reduce = 0%
2026-10-08 13:40:28,624 Stage-1 map = 100%, reduce = 100%
Ended Job = job_local1557575145_0009
Launching Job 2 out of 2
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):


Administrateur : Invite de commandes
MapReduce Jobs Launched:
Stage-Stage-1: HDFS Read: 408060376 HDFS Write: 23455828 SUCCESS
Stage-Stage-2: HDFS Read: 408060376 HDFS Write: 23455828 SUCCESS
Total MapReduce CPU Time Spent: 0 msec
OK
ville	nb_transactions	ca_total	panier_moyen
Lome	200348	1.838294035E9	9175.5
Kara	59635	5.509592675E8	9238.86
Sokode	54686	5.035592375E8	9208.19
Kpalime	45369	4.16192485E8	9173.5
Atakpame	44803	4.1125822E8	9179.26
Tsevie	35204	3.235216825E8	9189.91
Dapaong	34808	3.212253125E8	9228.49
Aneho	25147	2.3108381E8	9189.32
Time taken: 12.309 seconds, Fetched: 8 row(s)
WARNING: Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider using a different execution engine (i.e. spark, tez) or using Hive 1.X releases.
Query ID = root_20261008134030_3d3e83de-d32c-4c69-b2cf-74c713216881
Total jobs = 2
Launching Job 1 out of 2
Number of reduce tasks not specified. Estimated from input data size: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Job running in-process (local Hadoop)
2026-10-08 13:40:37,184 Stage-1 map = 0%, reduce = 0%
2026-10-08 13:40:42,235 Stage-1 map = 42%, reduce = 0%


Administrateur : Invite de commandes
2026-10-08 13:40:37,184 Stage-1 map = 0%, reduce = 0%
2026-10-08 13:40:42,235 Stage-1 map = 42%, reduce = 0%
2026-10-08 13:40:45,263 Stage-1 map = 56%, reduce = 0%
2026-10-08 13:40:46,269 Stage-1 map = 100%, reduce = 100%
Ended Job = job_local1817866902_0011
Launching Job 2 out of 2
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Job running in-process (local Hadoop)
2026-10-08 13:40:49,345 Stage-2 map = 0%, reduce = 0%
2026-10-08 13:40:50,350 Stage-2 map = 100%, reduce = 100%
Ended Job = job_local162860340_0012
MapReduce Jobs Launched:
Stage-Stage-1: HDFS Read: 412603108 HDFS Write: 23455828 SUCCESS
Stage-Stage-2: HDFS Read: 412603108 HDFS Write: 23455828 SUCCESS
Total MapReduce CPU Time Spent: 0 msec
OK
produit categorie   quantite_vendue ca_total
Powerbank   Electronique   76312   8.884057175E8
Souris Electronique   75158   5.8379907E8
Cle_USB Electronique   76416   5.194914925E8
Ecouteurs   Electronique   76360   4.8238596E8
Riz_5kg Alimentation   75953   4.7922964E8
Ramette_A4   Papeterie   76284   3.33786595E8
Cafe   Boissons   75481   1.8315708E8


Administrateur : Invite de commandes
MapReduce Jobs Launched:
Stage-Stage-1: HDFS Read: 416911856 HDFS Write: 23455828 SUCCESS
Stage-Stage-2: HDFS Read: 416911856 HDFS Write: 23455828 SUCCESS
Total MapReduce CPU Time Spent: 0 msec
OK
categorie   mode_paiement   nb_transactions quantite_vendue ca_total
Alimentation   Especes 45286   145022 3.1805418E8
Alimentation   TMoney 29573   94279   2.05282875E8
Alimentation   Flooz   23749   75840   1.6675616E8
Alimentation   Carte   11925   38238   8.34904125E7
Alimentation   Virement   8407   26742   5.67619325E7
Boissons   Especes 36225   115914 1.316108825E8
Boissons   TMoney 24162   77332   8.77346375E7
Boissons   Flooz   19123   61090   6.9024395E7
Boissons   Carte   9430   30140   3.4413155E7
Boissons   Virement   6718   21719   2.48273875E7
Electronique   Especes 35865   114918 9.335254075E8
Electronique   TMoney 23778   76073   6.19178375E8
Electronique   Flooz   19183   61563   5.005800225E8
Electronique   Carte   9553   30635   2.49310125E8
Electronique   Virement   6597   21057   1.7148831E8
Hygiene Especes 36222   116204 1.719431825E8
Hygiene TMoney 23890   76388   1.1328653E8
Hygiene Flooz   18786   60105   8.959912E7
Hygiene Carte   9368   30051   4.430596E7
Hygiene Virement   6525   20742   3.0964495E7
Papeterie   Especes 36372   116519 1.87242705E8
Papeterie   TMoney 23919   76753   1.2460376E8
Papeterie   Flooz   19298   61551   9.9015365E7
Papeterie   Carte   9532   30640   4.9678285E7



4. TP détaillé en Python (PyHive)

Une fois le cluster démarré et les tables Hive créées, l'analyse peut être pilotée directement depuis Python grâce à la bibliothèque PyHive, qui se connecte à HiveServer2 (port 10000, exposé sur l'hôte Windows par docker-compose).

Étape 1 : Installer les dépendances Python sur la machine hôte

```
pip install pyhive thrift future pandas matplotlib
```

Important : ne pas installer pyhive[hive] ni sasl. Le module sasl demande un compilateur C++ sous Windows (erreur « Microsoft Visual C++ 14.0 or greater is required »). Le serveur HiveServer2 est configuré en mode NOSASL dans hadoop-hive.env, ce qui rend ce module inutile.

Étape 2 : Se connecter à Hive depuis Python

```
from pyhive import hive
import pandas as pd
conn = hive.Connection(host='localhost', port=10000,
                       username='root', database='default',
                       auth='NOSASL')
curseur = conn.cursor()
curseur.execute('SELECT * FROM transactions LIMIT 10')
df = pd.DataFrame(curseur.fetchall(), columns=[c[0] for c in curseur.description])
print(df)
```

Étape 3 : Requêtes d'analyse demandées

1. Chiffre d'affaires mensuel et annuel
2. Chiffre d'affaires et panier moyen par ville
3. Top 10 des produits
4. Ventes par catégorie et mode de paiement
5. Impact des promotions
6. Activité par jour de la semaine
7. Comparaison avec les résultats MapReduce (5 jobs du Groupe 3)

Exécution :

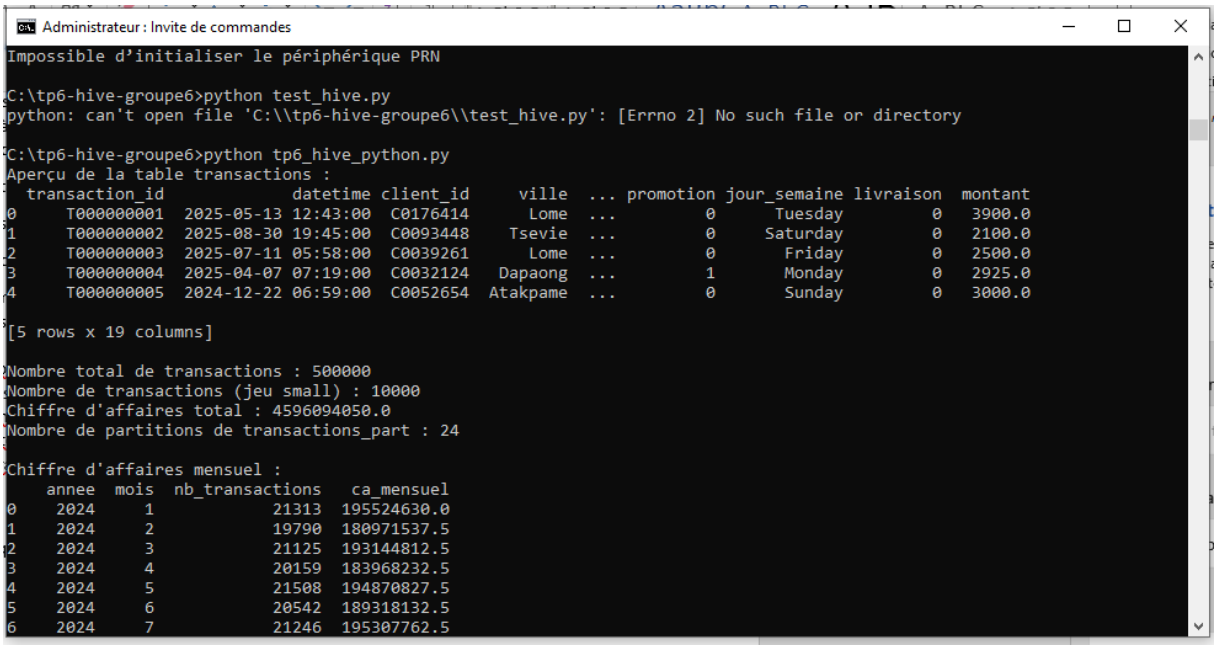
```
python tp6_hive_python.py
```

Combien de transactions contient le jeu de données (jeu medium) ?

Requête HiveQL :

```
SELECT COUNT(*) AS nb_transactions FROM transactions;
```

Résultat :

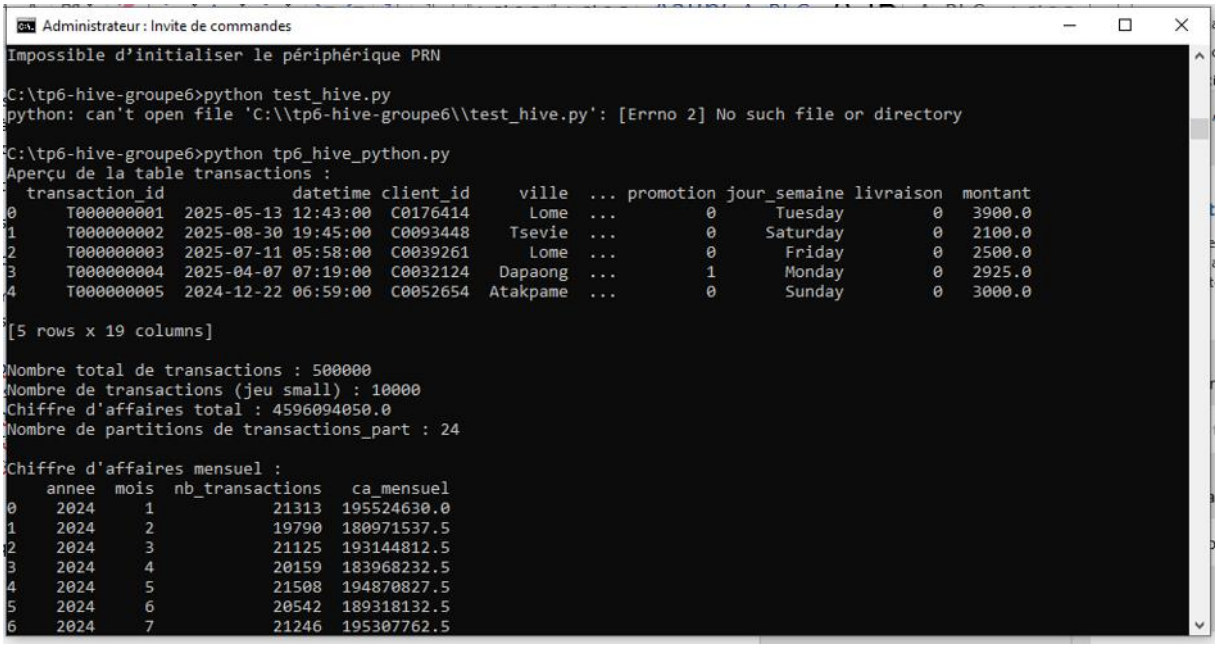


Quel est le chiffre d'affaires total réalisé ?

Requête HiveQL

```
SELECT ROUND(SUM(montant), 2) AS ca_total FROM transactions;
```

Résultat :



Script :HiveQL complet

```
--
================================================================
=====
-- Exposé 6 - Hive et Data Warehouse distribué (Groupe 6)
-- Script HiveQL : création des tables externes + partitionnement + requêtes d'analyse
--
================================================================
=====


SET hive.cli.print.header=true;
SET hive.exec.dynamic.partition=true;
SET hive.exec.dynamic.partition.mode=nonstrict;


-- 1. Création des tables externes pointant vers HDFS
--    transactions       : jeu medium (500 000 lignes) - développement et validation
--    transactions_small : jeu small  (10 000 lignes)  - tests rapides
CREATE EXTERNAL TABLE IF NOT EXISTS transactions (
   transaction_id STRING,
   datetime       STRING,
   client_id      STRING,
   ville          STRING,
   magasin_id     STRING,
   categorie      STRING,
   produit        STRING,
   quantite       INT,
   prix_unitaire  DOUBLE,
   remise         DOUBLE,
   mode_paiement  STRING,
```

```
    age_client     INT,
    type_client    STRING,
    stock_avant    INT,
    stock_apres    INT,
    promotion      INT,
    jour_semaine   STRING,
    livraison      INT,
    montant        DOUBLE
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION '/datalake/raw/transactions/medium'
TBLPROPERTIES ("skip.header.line.count"="1");

CREATE EXTERNAL TABLE IF NOT EXISTS transactions_small (
    transaction_id STRING,
    datetime       STRING,
    client_id      STRING,
    ville          STRING,
    magasin_id     STRING,
    categorie      STRING,
    produit        STRING,
    quantite       INT,
    prix_unitaire  DOUBLE,
    remise         DOUBLE,
    mode_paiement  STRING,
```

```
    age_client     INT,
    type_client    STRING,
    stock_avant    INT,
    stock_apres    INT,
    promotion      INT,
    jour_semaine   STRING,
    livraison      INT,
    montant        DOUBLE
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION '/datalake/raw/transactions/small'
TBLPROPERTIES ("skip.header.line.count"="1");

-- 2. Vérification rapide
SELECT * FROM transactions LIMIT 10;
SELECT COUNT(*) AS nb_transactions FROM transactions;
SELECT COUNT(*) AS nb_transactions_small FROM transactions_small;
SELECT ROUND(SUM(montant), 2) AS ca_total FROM transactions;

-- 3. Organisation partitionnée par (annee, mois) au format ORC
--    24 partitions (2 ans x 12 mois) : le filtre WHERE annee=... AND mois=...
--    ne lit qu'une partition (partition pruning)
CREATE TABLE IF NOT EXISTS transactions_part (
    transaction_id STRING,
    datetime       STRING,
```

```
    client_id       STRING,
    ville           STRING,
    magasin_id      STRING,
    categorie       STRING,
    produit         STRING,
    quantite        INT,
    prix_unitaire   DOUBLE,
    remise          DOUBLE,
    mode_paiement   STRING,
    age_client      INT,
    type_client     STRING,
    stock_avant     INT,
    stock_apres     INT,
    promotion       INT,
    jour_semaine    STRING,
    livraison       INT,
    montant         DOUBLE
)
PARTITIONED BY (annee INT, mois INT)
STORED AS ORC;

INSERT OVERWRITE TABLE transactions_part PARTITION (annee, mois)
SELECT transaction_id, datetime, client_id, ville, magasin_id, categorie, produit,
       quantite, prix_unitaire, remise, mode_paiement, age_client, type_client,
       stock_avant, stock_apres, promotion, jour_semaine, livraison, montant,
       CAST(SUBSTR(datetime, 1, 4) AS INT) AS annee,
       CAST(SUBSTR(datetime, 6, 2) AS INT) AS mois
```

```
FROM transactions;


SHOW PARTITIONS transactions_part;


-- 4. Q1a - Chiffre d'affaires mensuel
SELECT annee, mois,
      COUNT(*) AS nb_transactions,
      ROUND(SUM(montant), 2) AS ca_mensuel
FROM transactions_part
GROUP BY annee, mois
ORDER BY annee, mois;


-- 5. Q1b - Chiffre d'affaires annuel
SELECT annee,
      COUNT(*) AS nb_transactions,
      ROUND(SUM(montant), 2) AS ca_annuel
FROM transactions_part
GROUP BY annee
ORDER BY annee;


-- 6. Q2 - Chiffre d'affaires et panier moyen par ville
--    (une ligne = une transaction, donc panier moyen = AVG(montant))
SELECT ville,
      COUNT(*) AS nb_transactions,
      ROUND(SUM(montant), 2) AS ca_total,
      ROUND(AVG(montant), 2) AS panier_moyen
FROM transactions_part
```

```
GROUP BY ville
ORDER BY ca_total DESC;

-- 7. Q3 - Top 10 des produits (par chiffre d'affaires)
SELECT produit, categorie,
    SUM(quantite) AS quantite_vendue,
    ROUND(SUM(montant), 2) AS ca_total
FROM transactions_part
GROUP BY produit, categorie
ORDER BY ca_total DESC
LIMIT 10;

-- 8. Q4 - Ventes par catégorie et par mode de paiement
SELECT categorie, mode_paiement,
    COUNT(*) AS nb_transactions,
    SUM(quantite) AS quantite_vendue,
    ROUND(SUM(montant), 2) AS ca_total
FROM transactions_part
GROUP BY categorie, mode_paiement
ORDER BY categorie, ca_total DESC;

-- 9. Q5 - Impact des promotions (promotion = 1) par rapport à l'absence de promotion (0)
--    cout_remises = CA brut - CA net
SELECT promotion,
    COUNT(*) AS nb_transactions,
    ROUND(SUM(montant), 2) AS ca_net,
    ROUND(SUM(quantite * prix_unitaire), 2) AS ca_brut,
```

```
    ROUND(SUM(quantite * prix_unitaire) - SUM(montant), 2) AS cout_remises,
    ROUND(AVG(montant), 2) AS panier_moyen,
    ROUND(AVG(quantite), 3) AS quantite_moyenne,
    ROUND(AVG(remise), 4) AS remise_moyenne
FROM transactions_part
GROUP BY promotion
ORDER BY promotion;


-- 10. Q6 - Activité par jour de la semaine (ordre calendaire)
--   La colonne 'ordre' doit figurer dans le SELECT externe : Hive 2.3 refuse un ORDER BY
--   sur une colonne absente du SELECT de la requête qui trie.
SELECT jour_semaine, nb_transactions, ca_total, panier_moyen, ordre
FROM (
  SELECT jour_semaine,
      COUNT(*) AS nb_transactions,
      ROUND(SUM(montant), 2) AS ca_total,
      ROUND(AVG(montant), 2) AS panier_moyen,
      CASE jour_semaine
        WHEN 'Monday' THEN 1 WHEN 'Tuesday' THEN 2 WHEN 'Wednesday' THEN 3
        WHEN 'Thursday' THEN 4 WHEN 'Friday' THEN 5 WHEN 'Saturday' THEN 6
        ELSE 7 END AS ordre
  FROM transactions_part
  GROUP BY jour_semaine
) t
ORDER BY ordre;
```

```
-- 11. Q7 - Équivalents des 5 jobs MapReduce (Exposé 3 / Exposé 4)
--     a) nombre de transactions par ville
SELECT ville, COUNT(*) AS nb_transactions
FROM transactions
GROUP BY ville
ORDER BY ville;

--     b) chiffre d'affaires par ville : somme de q x p x (1 - r)
SELECT ville, ROUND(SUM(quantite * prix_unitaire * (1 - remise)), 2) AS ca
FROM transactions
GROUP BY ville
ORDER BY ville;

--     c) quantité vendue par produit
SELECT produit, SUM(quantite) AS quantite_vendue
FROM transactions
GROUP BY produit
ORDER BY produit;

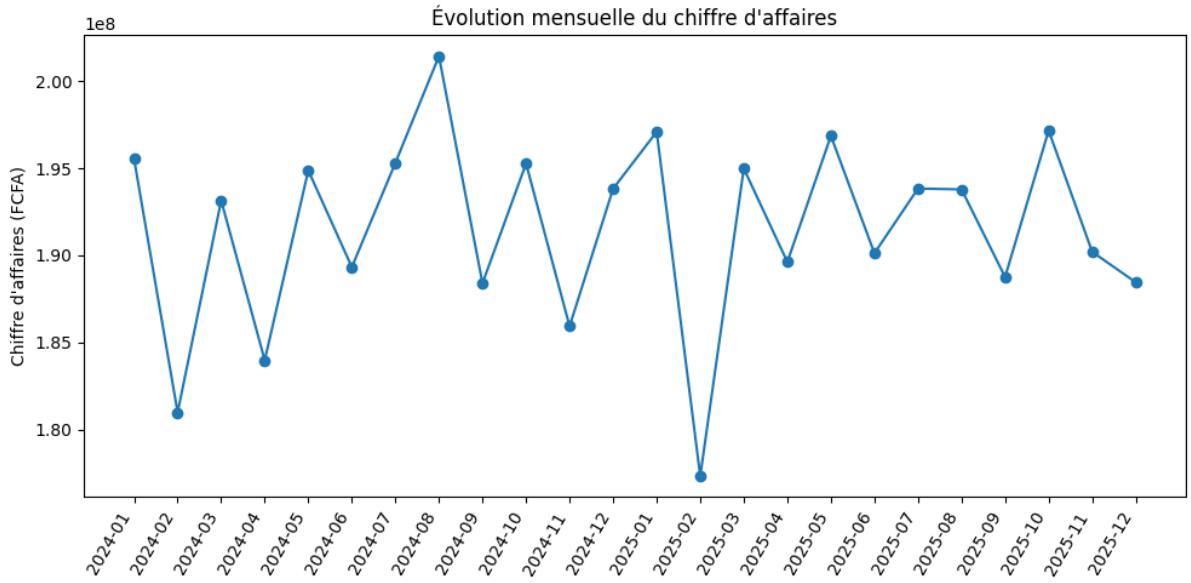
--     d) chiffre d'affaires par catégorie
SELECT categorie, ROUND(SUM(quantite * prix_unitaire * (1 - remise)), 2) AS ca
FROM transactions
GROUP BY categorie
ORDER BY categorie;

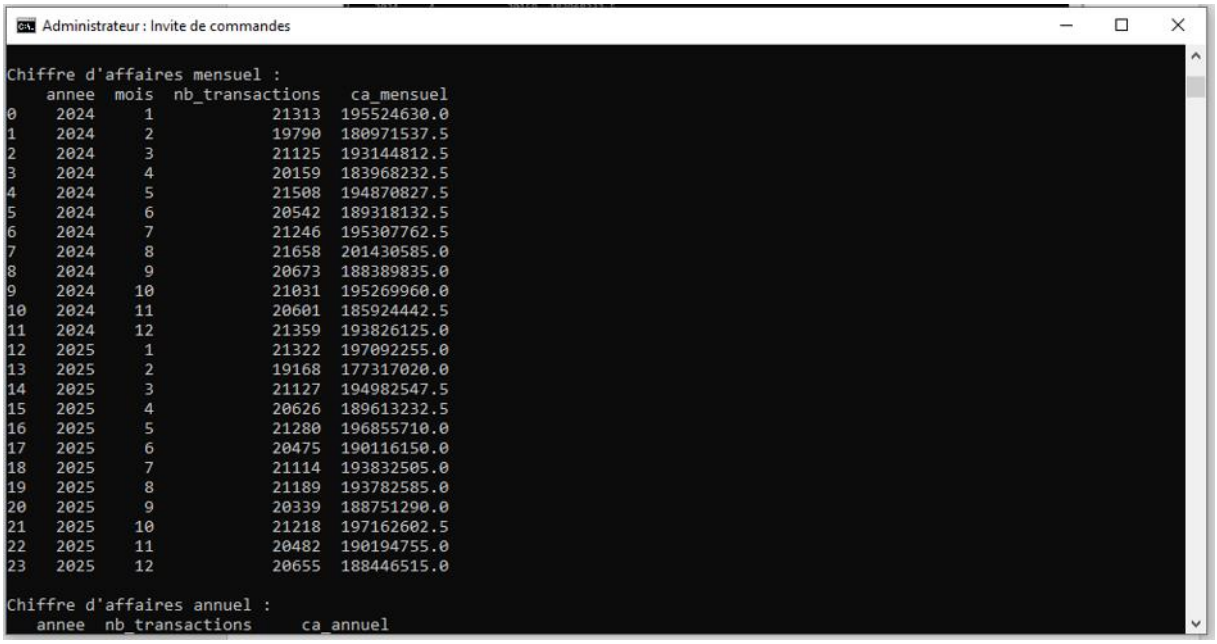
--     e) nombre de transactions par mode de paiement
SELECT mode_paiement, COUNT(*) AS nb_transactions
```

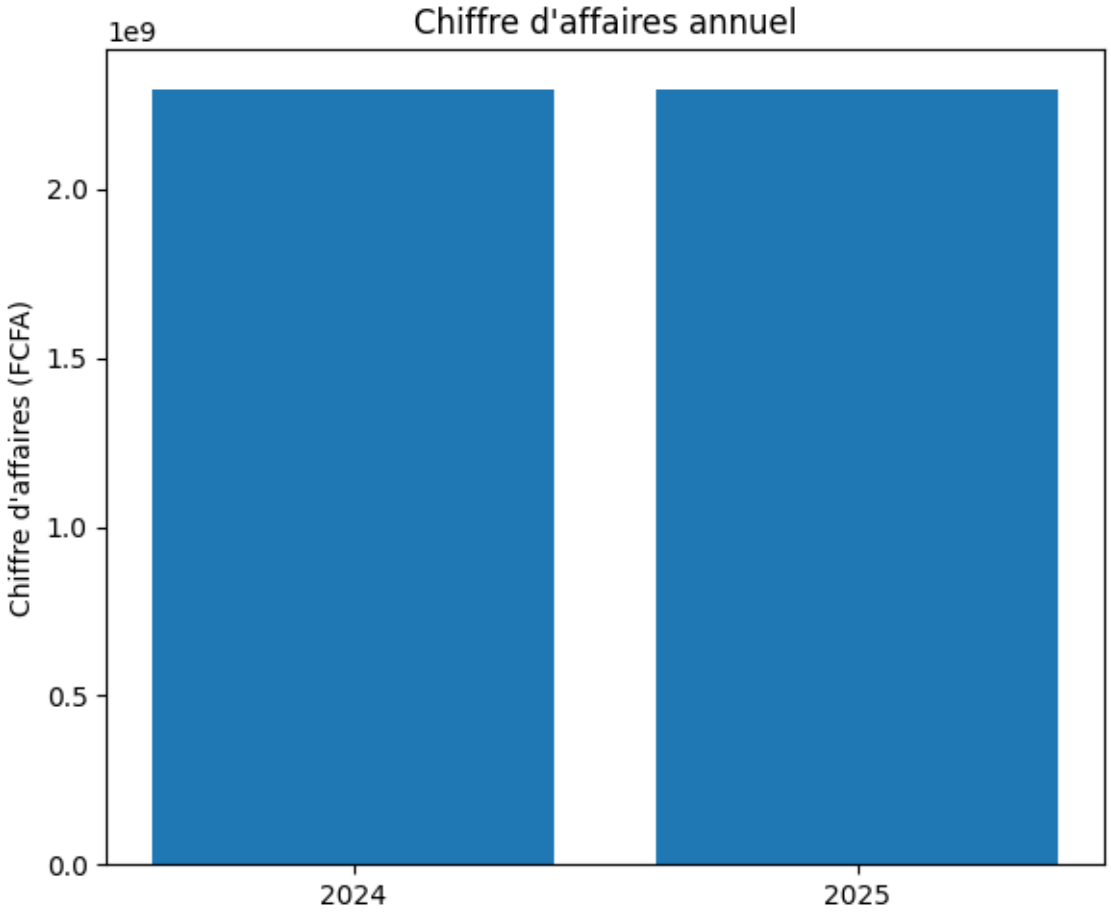
```
FROM transactions
GROUP BY mode_paiement
ORDER BY mode_paiement;
```

Resultat du TP

1. Chiffre d'affaires mensuel et annuel







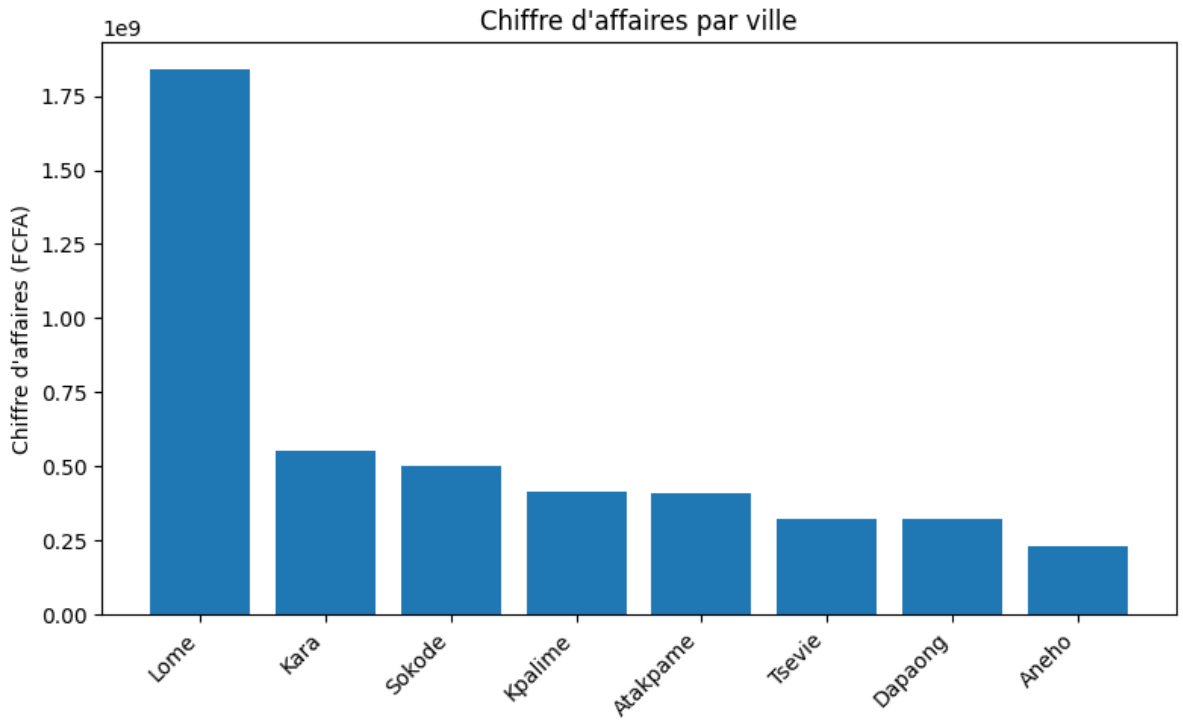
Administrateur : Invite de commandes

```
1    2024     2            19790  180971537.5
2    2024     3            21125  193144812.5
3    2024     4            20159  183968232.5
4    2024     5            21508  194870827.5
5    2024     6            20542  189318132.5
6    2024     7            21246  195307762.5
7    2024     8            21658  201430585.0
8    2024     9            20673  188389835.0
9    2024    10            21031  195269960.0
10   2024    11            20601  185924442.5
11   2024    12            21359  193826125.0
12   2025     1            21322  197092255.0
13   2025     2            19168  177317020.0
14   2025     3            21127  194982547.5
15   2025     4            20626  189613232.5
16   2025     5            21280  196855710.0
17   2025     6            20475  190116150.0
18   2025     7            21114  193832505.0
19   2025     8            21189  193782585.0
20   2025     9            20339  188751290.0
21   2025    10            21218  197162602.5
22   2025    11            20482  190194755.0
23   2025    12            20655  188446515.0

Chiffre d'affaires annuel :
   annee  nb_transactions     ca_annuel
0   2024           251005  2.297947e+09
1   2025           248995  2.298147e+09

Chiffre d'affaires et panier moyen par ville :
```

2. Chiffre d'affaires et panier moyen par ville



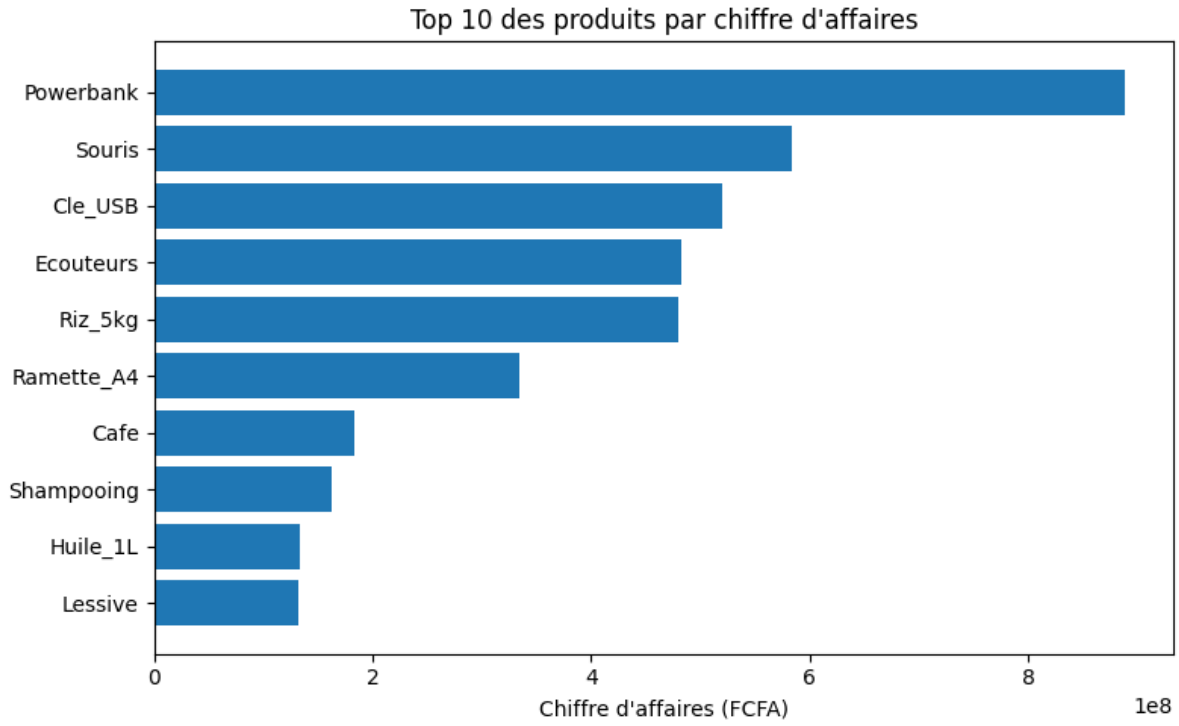

```
Administrateur : Invite de commandes
22   2025    11           20482  190194755.0
23   2025    12           20655  188446515.0

Chiffre d'affaires annuel :
   annee  nb_transactions     ca_annuel
0   2024           251005  2.297947e+09
1   2025           248995  2.298147e+09

Chiffre d'affaires et panier moyen par ville :
      ville  nb_transactions      ca_total  panier_moyen
0      Lome           200348  1.838294e+09       9175.50
1      Kara            59635  5.509593e+08       9238.86
2    Sokode            54686  5.035592e+08       9208.19
3   Kpalime            45369  4.161925e+08       9173.50
4  Atakpame            44803  4.112582e+08       9179.26
5    Tsevie            35204  3.235217e+08       9189.91
6   Dapaong            34808  3.212253e+08       9228.49
7     Aneho            25147  2.310838e+08       9189.32

Top 10 des produits (par chiffre d'affaires) :
      produit     categorie  quantite_vendue     ca_total
0   Powerbank  Electronique            76312  888405717.5
1      Souris  Electronique            75158  583799070.0
2     Cle_USB  Electronique            76416  519491492.5
3   Ecouteurs  Electronique            76360  482385960.0
4     Riz_5kg  Alimentation            75953  479229640.0
5  Ramette_A4     Papeterie            76284  333786595.0
6        Cafe      Boissons            75481  183157080.0
7  Shampooing       Hygiene            76193  162800165.0
8    Huile_1L  Alimentation            76350  133519407.5
```

3. Top 10 des produits

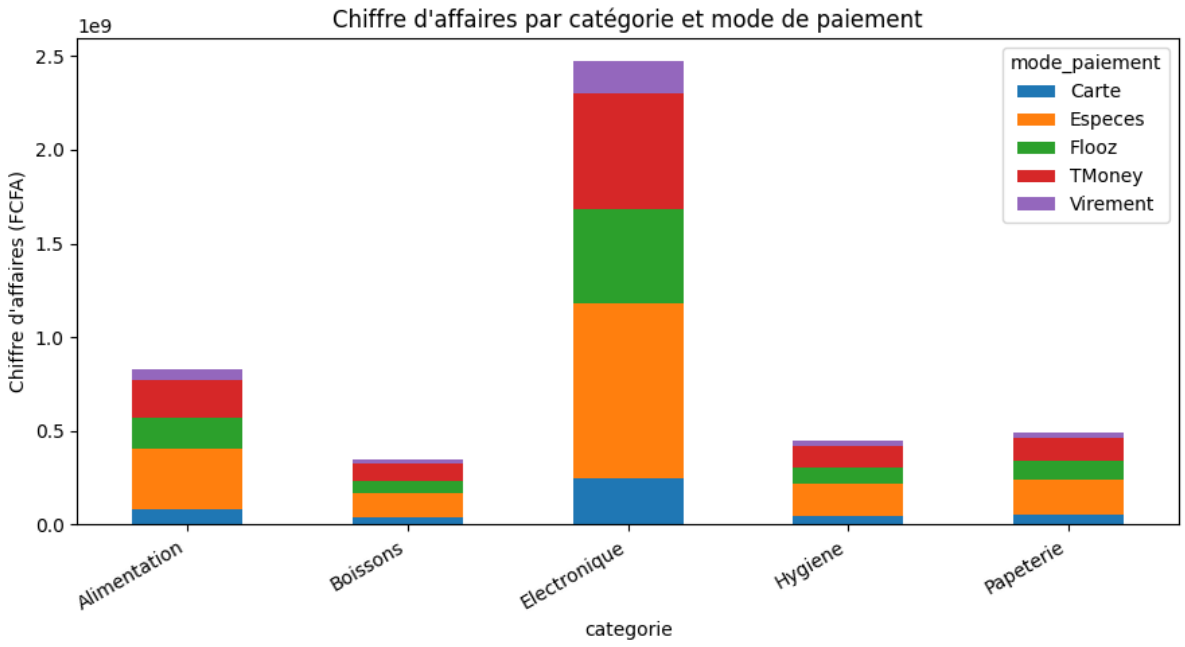


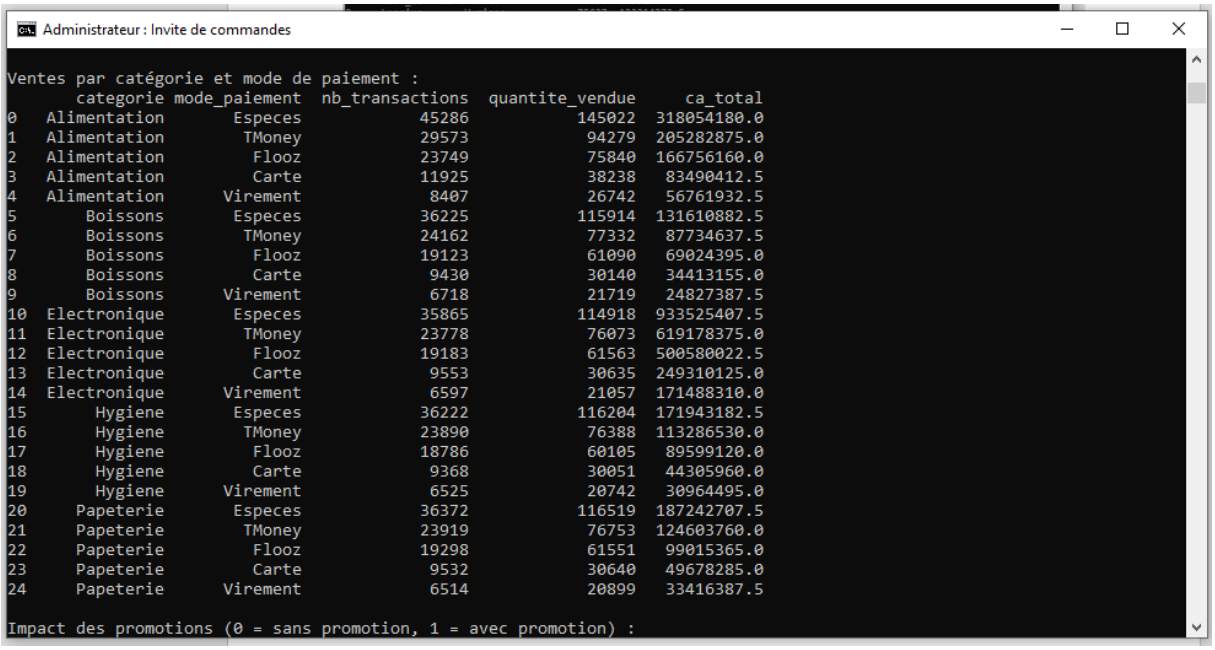
Administrateur : Invite de commandes

```
Top 10 des produits (par chiffre d'affaires) :
      produit     categorie  quantite_vendue     ca_total
0   Powerbank  Electronique            76312  888405717.5
1      Souris  Electronique            75158  583799070.0
2     Cle_USB  Electronique            76416  519491492.5
3   Ecouteurs  Electronique            76360  482385960.0
4     Riz_5kg  Alimentation            75953  479229640.0
5  Ramette_A4     Papeterie            76284  333786595.0
6        Cafe      Boissons            75481  183157080.0
7  Shampooing       Hygiene            76193  162800165.0
8    Huile_1L  Alimentation            76350  133519407.5
9     Lessive       Hygiene            75637  132314372.5

Ventes par catégorie et mode de paiement :
       categorie mode_paiement  nb_transactions  quantite_vendue     ca_total
0   Alimentation       Especes            45286           145022  318054180.0
1   Alimentation        TMoney            29573            94279  205282875.0
2   Alimentation         Flooz            23749            75840  166756160.0
3   Alimentation         Carte            11925            38238   83490412.5
4   Alimentation      Virement             8407            26742   56761932.5
5       Boissons       Especes            36225           115914  131610882.5
6       Boissons        TMoney            24162            77332   87734637.5
7       Boissons         Flooz            19123            61090   69024395.0
8       Boissons         Carte             9430            30140   34413155.0
9       Boissons      Virement             6718            21719   24827387.5
10  Electronique       Especes            35865           114918  933525407.5
11  Electronique        TMoney            23778            76073  619178375.0
12  Electronique         Flooz            19183            61563  500580022.5
13  Electronique         Carte             9553            30635  249310125.0
```

4. Ventes par catégorie et mode de paiement



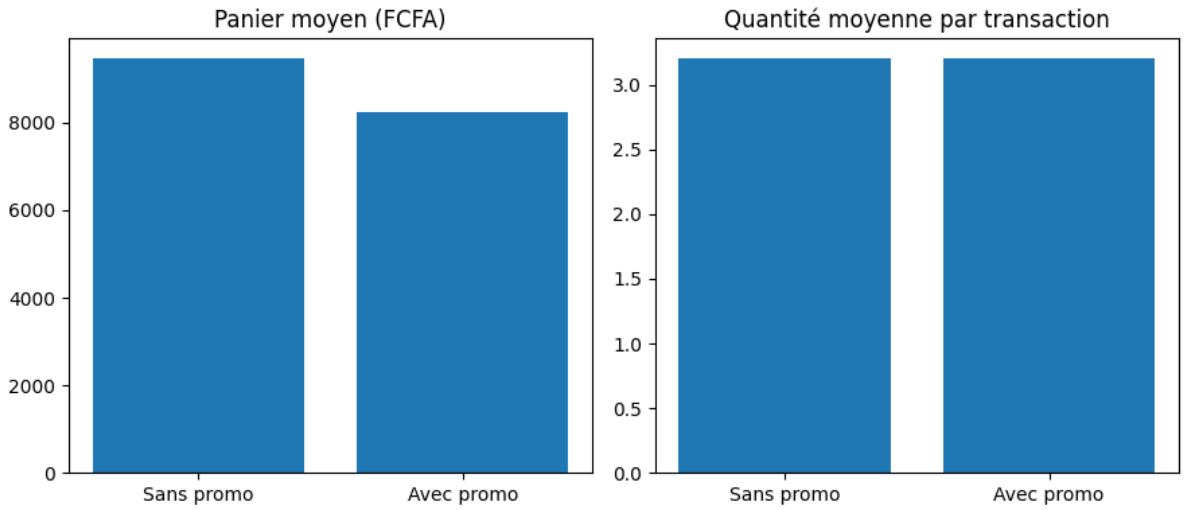


Administrateur : Invite de commandes

```
Ventes par catégorie et mode de paiement :
       categorie mode_paiement  nb_transactions  quantite_vendue     ca_total
0   Alimentation       Especes            45286           145022  318054180.0
1   Alimentation        TMoney            29573            94279  205282875.0
2   Alimentation         Flooz            23749            75840  166756160.0
3   Alimentation         Carte            11925            38238   83490412.5
4   Alimentation     Virement             8407            26742   56761932.5
5       Boissons       Especes            36225           115914  131610882.5
6       Boissons        TMoney            24162            77332   87734637.5
7       Boissons         Flooz            19123            61090   69024395.0
8       Boissons         Carte             9430            30140   34413155.0
9       Boissons     Virement             6718            21719   24827387.5
10  Electronique       Especes            35865           114918  933525407.5
11  Electronique        TMoney            23778            76073  619178375.0
12  Electronique         Flooz            19183            61563  500580022.5
13  Electronique         Carte             9553            30635  249310125.0
14  Electronique     Virement             6597            21057  171488310.0
15       Hygiene       Especes            36222           116204  171943182.5
16       Hygiene        TMoney            23890            76388  113286530.0
17       Hygiene         Flooz            18786            60105   89599120.0
18       Hygiene         Carte             9368            30051   44305960.0
19       Hygiene     Virement             6525            20742   30964495.0
20     Papeterie       Especes            36372           116519  187242707.5
21     Papeterie        TMoney            23919            76753  124603760.0
22     Papeterie         Flooz            19298            61551   99015365.0
23     Papeterie         Carte             9532            30640   49678285.0
24     Papeterie     Virement             6514            20899   33416387.5

Impact des promotions (0 = sans promotion, 1 = avec promotion) :
```

5. Impact des promotions




```
Administrateur : Invite de commandes

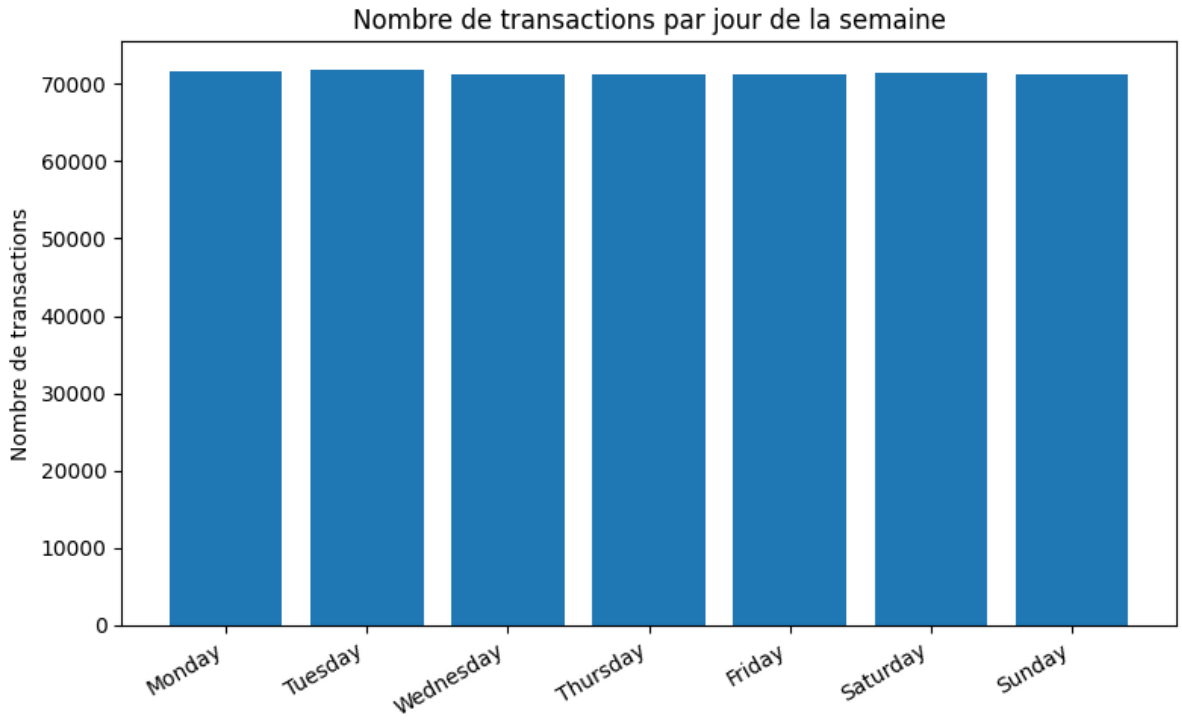
Impact des promotions (0 = sans promotion, 1 = avec promotion) :
   promotion  nb_transactions        ca_net       ca_brut  cout_remises  panier_moyen  quantite_moyenne  remise_moyenne
0          0           390322  3.693298e+09  3.693298e+09           0.0       9462.18             3.201          0.0000
1          1           109678  9.027956e+08  1.039648e+09   136852450.0       8231.33             3.199          0.1317

Activité par jour de la semaine :
  jour_semaine  nb_transactions     ca_total  panier_moyen
0       Monday            71694  667010575.0       9303.58
1      Tuesday            71870  655747850.0       9124.08
2    Wednesday            71199  651217245.0       9146.44
3     Thursday            71317  652252710.0       9145.82
4       Friday            71274  654310700.0       9180.22
5     Saturday            71477  657024605.0       9192.11
6       Sunday            71169  658530365.0       9253.05

Comparaison Hive / MapReduce :
                      job  nb_cles_hive  nb_cles_reference  ecart_max       origine_reference statut
0     transactions_par_ville             8                  8        0.0  reconstruite localement   PASS
1              ca_par_ville             8                  8        0.0  reconstruite localement   PASS
2       quantite_par_produit            21                 21        0.0  reconstruite localement   PASS
3          ca_par_categorie             5                  5        0.0  reconstruite localement   PASS
4  transactions_par_paiement             5                  5        0.0  reconstruite localement   PASS

Temps d'exécution des requêtes Hive :
                        requete  duree_s
0                Q1a_ca_mensuel     19.1
1                 Q1b_ca_annuel     12.7
2            Q2_ca_panier_ville     23.6
3             Q3_top10_produits     15.3
```

6. activité par jour de semaine ;



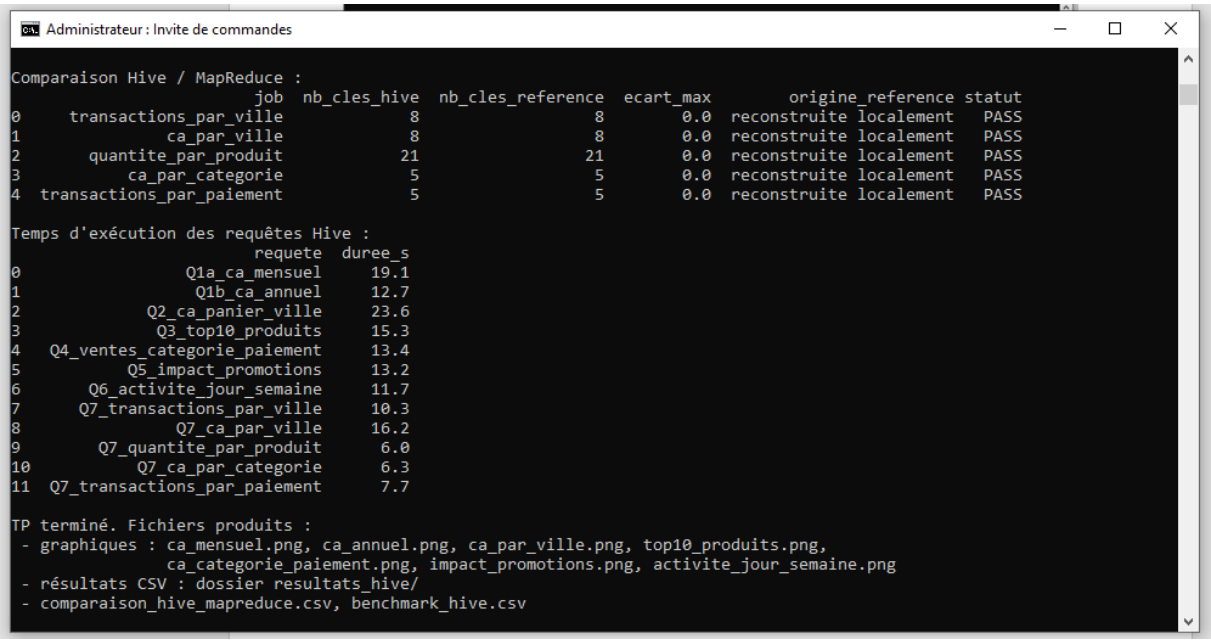

```
Administrateur : Invite de commandes

Activité par jour de la semaine :
  jour_semaine  nb_transactions     ca_total  panier_moyen
0       Monday            71694  667010575.0       9303.58
1      Tuesday            71870  655747850.0       9124.08
2    Wednesday            71199  651217245.0       9146.44
3     Thursday            71317  652252710.0       9145.82
4       Friday            71274  654310700.0       9180.22
5     Saturday            71477  657024605.0       9192.11
6       Sunday            71169  658530365.0       9253.05

Comparaison Hive / MapReduce :
                     job  nb_cles_hive  nb_cles_reference  ecart_max        origine_reference statut
0     transactions_par_ville             8                  8        0.0  reconstruite localement   PASS
1              ca_par_ville             8                  8        0.0  reconstruite localement   PASS
2       quantite_par_produit            21                 21        0.0  reconstruite localement   PASS
3           ca_par_categorie             5                  5        0.0  reconstruite localement   PASS
4  transactions_par_paiement             5                  5        0.0  reconstruite localement   PASS

Temps d'exécution des requêtes Hive :
                       requete  duree_s
0                Q1a_ca_mensuel     19.1
1                 Q1b_ca_annuel     12.7
2            Q2_ca_panier_ville     23.6
3             Q3_top10_produits     15.3
4   Q4_ventes_categorie_paiement     13.4
5           Q5_impact_promotions     13.2
6       Q6_activite_jour_semaine     11.7
7      Q7_transactions_par_ville     10.3
8               Q7_ca_par_ville     16.2
```

7. Comparaison avec les résultats MapReduce.



- nb_cles_hive = nb_cles_reference : même nombre de clés des deux côtés, soit 8 villes, 21 produits, 5 catégories et 5 modes de paiement. Aucune clé ne manque ni n'est en trop.
- ecart_max = 0.0 : la plus grande différence de valeur est nulle, donc les chiffres sont identiques (tolérance de 0,01 FCFA).
- PASS : mêmes clés et écart inférieur à la tolérance, pour les 5 jobs. Hive et la logique MapReduce donnent exactement les mêmes résultats.

Les cinq indicateurs calculés avec Hive sont identiques, clé par clé, à ceux de la logique MapReduce, avec un écart maximal nul. Les divergences possibles (séparateur, en-tête, types, valeurs manquantes, arrondis, doublons, logique analytique) ont été contrôlées et aucune n'apparaît. La référence MapReduce a été reconstruite localement faute des sorties du Groupe 4 : c'est une anomalie documentée. Chaque requête prend de 6 à 24 secondes. Hive tourne ici en MapReduce local, donc une grande part de ce temps est le démarrage du job, pas le calcul. son intérêt est de passer à l'échelle et d'écrire du SQL à la place de code mapper.py et reducer.py. N'extrapole pas ces durées à 5 millions de lignes sans les mesurer.

Dépannage courant

Erreur de connexion PyHive sur le port 10000	Attendre 1-2 minutes après docker compose up (le Metastore démarre après la base Postgres).
« docker : command not found »	Docker Desktop n'est pas démarré ou pas dans le PATH relancer l'application.

Conteneurs qui redémarrent en boucle	Vérifier les ressources allouées (RAM/CPU) dans Docker Desktop.
« Table not found » dans Hive	Vérifier que les fichiers ont bien été chargés dans /datalake/raw/transactions/small et /medium avant de créer les tables.
COUNT (*) donne 510 000 au lieu de 500 000	Les fichiers small et medium se trouvent dans le même répertoire LOCATION ; un répertoire par fichier.
Colonnes numériques à NULL ou ligne d'en-tête lue comme une donnée	Vérifier le séparateur (',') et la propriété skip.header.line.count.
« Java heap space » pendant l'INSERT OVERWRITE de la table partitionnée	Augmenter HADOOP_HEAPSIZE (docker-compose.yml, service hive-server) et la mémoire allouée à Docker Desktop.
Erreur « Failed building wheel for sasl » / « Microsoft Visual C++ 14.0 or greater is required »	Ne pas installer sasl ; utiliser pip install pyhive thrift future pandas matplotlib (sans [hive]) et auth='NOSASL' dans la connexion (HiveServer2 est en NOSASL).
« TSocket read 0 bytes » à la connexion PyHive	HiveServer2 n'écoute pas encore (le port 10000 est ouvert par Docker mais le service démarre 1 à 3 minutes après un démarrage ou un --force-recreate, le metastore devant démarrer d'abord). Le script réessaie automatiquement toutes les 10 secondes (30 essais, 15 secondes maximum par essai pour ne jamais rester bloqué). Si le terminal semble figé sans message, arrêter avec Ctrl+C et lancer les commandes de diagnostic ci-dessous. Si l'échec persiste ; docker ps (hive-server doit être Up), docker logs hive-server --tail 40, et vérifier que NOSASL est bien appliqué avec docker exec hive-server sh -c "grep -r -A1 server2.authentication /opt/hive/conf /etc/hive" ; sinon recréer le service avec docker compose up -d --force-recreate hive-server.

« Invalid table alias or column reference 'ordre' »	Ancienne version du script Hive 2.3 exige que la colonne ordre figure dans le SELECT externe de la requête Q6 (corrigé dans cette version).
Réinitialiser le cluster	Docker compose down -v (supprime aussi les volumes HDFS et metastore), puis reprendre à l'étape 3